جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY

IS311 Web Development

Week 9 - Lecture 10

Dr. Abbas Malik
amaalik@psu.edu.sa

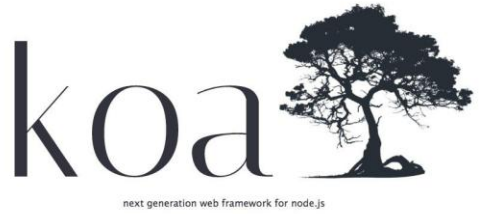
```
const http = require('http');
const Adder = require('../events/adder');
const adder = new Adder();

const serverObj = http.createServer(
    (request, response) => { // callback function
        if(request.url === '/'){
            response.write('Hello World from Node');
            response.end();
        }
        if(request.url === '/add'){
            adder.on('addedEvent', function(){
                response.write(`Sum of numbers is ${adder.sum}`);
                response.end();
            });
            adder.add(34, 56);
        }
    }
);
serverObj.listen(3000);
console.log('Listening on port 3000...');
```

Web Apps

- Can be developed using Node.js only
- But a framework make it easier to maintain, manage and code a web application using Node.js

Frameworks

- Web apps can be quickly and efficiently built
- Streamline development activities



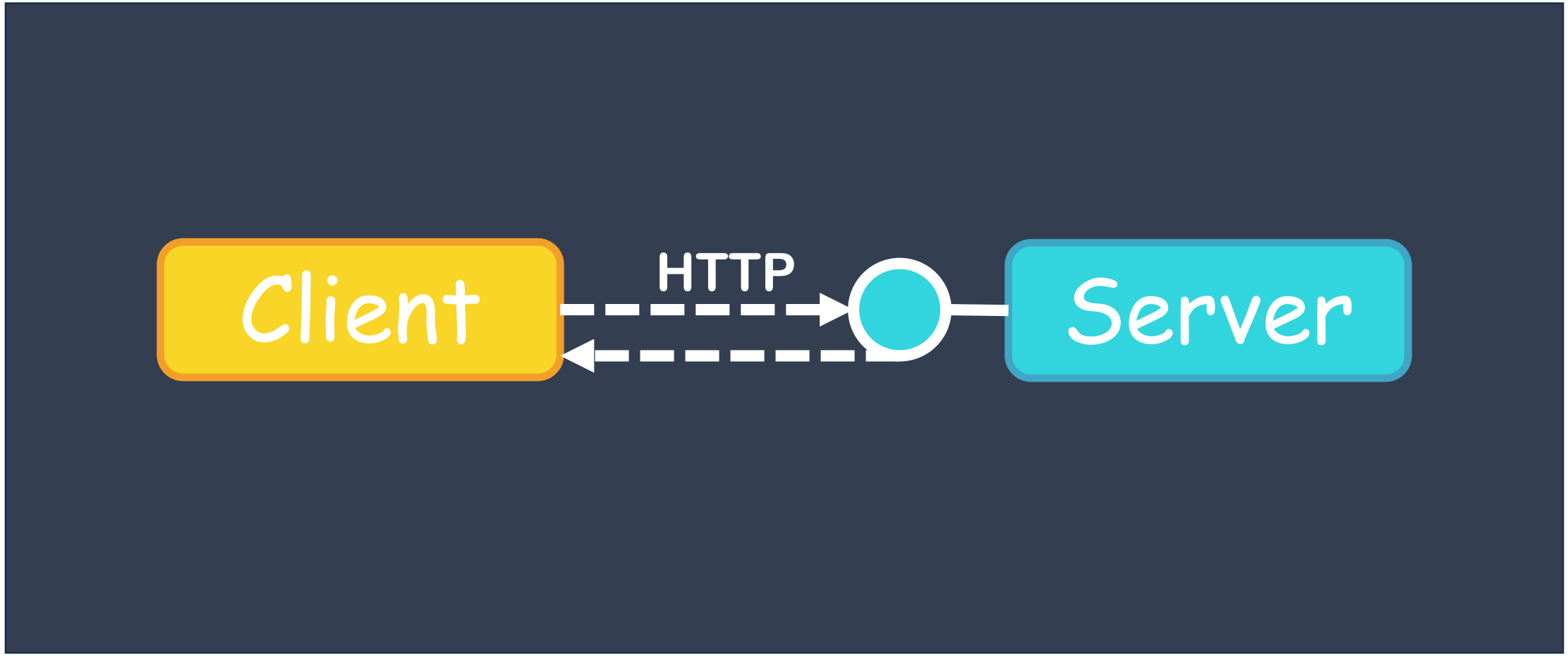
- Lightweight and adaptable
- Easy development
- Well-documented
- Generate HTML pages dynamically
- MVC – Model-View-Controller Design pattern
- Event-driven nature (callbacks)

HyperText Transfer Protocol

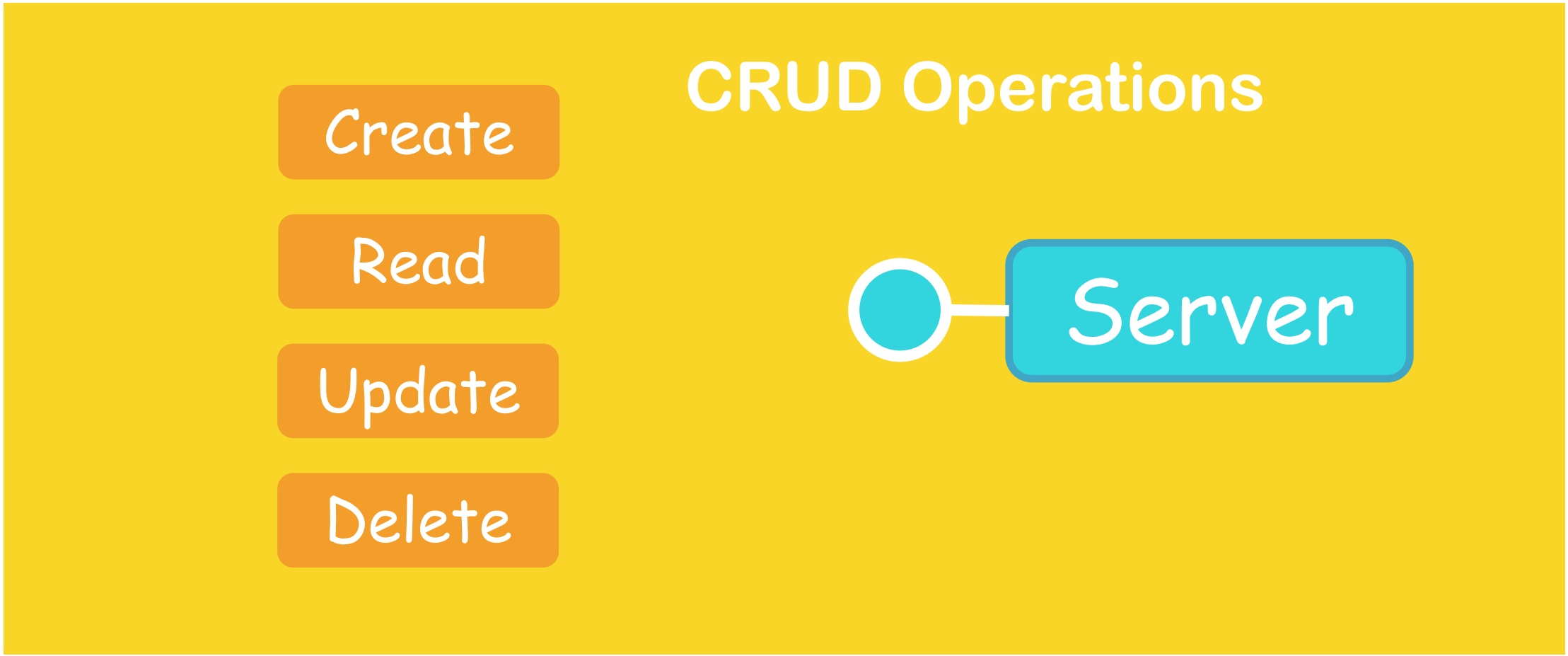
- Request and response stateless protocol

1. Client opens connection to server
2. Cient sends HTTP request to server
3. Server receives and process request
4. Server sends back the response
5. Connection to server is closed

Client - Server Arcitecture



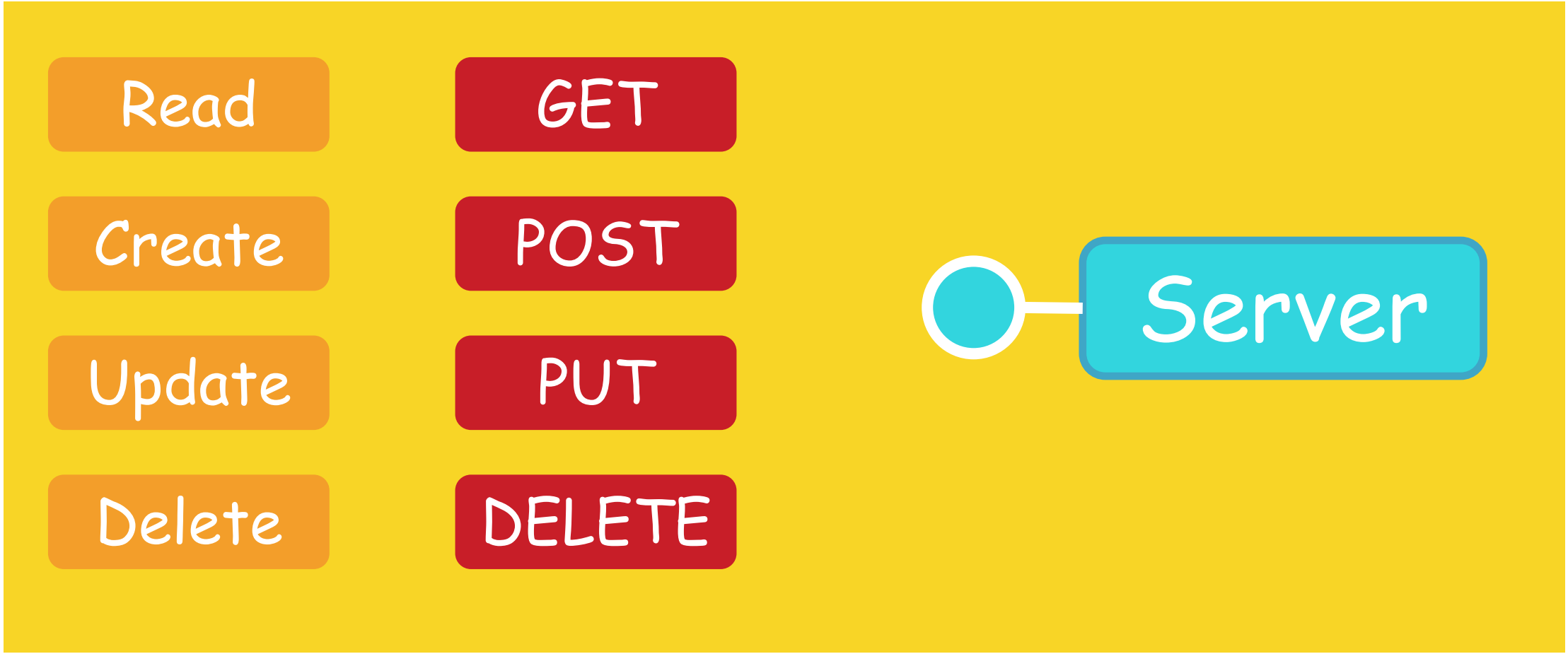
Representational State Transfer REST



Manage Students at PSU-CCIS

http://www.psu.edu/ccis/data/students

HTTP Methods



Requst and Response – Read Data

Request

GET /data/students

Response

```
[
   {“id”: 1, “name”: “Abbas”},
   {“id”: 2, “name”: “John”},
   …
]
```

Requst and Response – Read Data

Request

GET /data/students/1

Response

{“id”: 1, “name”: “Abbas”}

Requst and Response – Update Data

Request

PUT /data/students/1

{“name”: “M G Abbas”}

Response

```
{
   “id”: 1,
   “name”: “M G Abbas”
}
```

Requst and Response – Delete Data

Request

DELETE /data/students/1

Response

```
{
   “result”: “Success”
}
```

Requst and Response – Add Data

Request

POST /data/students/

```
{
   “name”: “Moeez”
}
```

Response

```
{
   “id”: 3,
   “name”: “Moeez”
}
```

Requst and Response - HTTP Request

HTTP Request

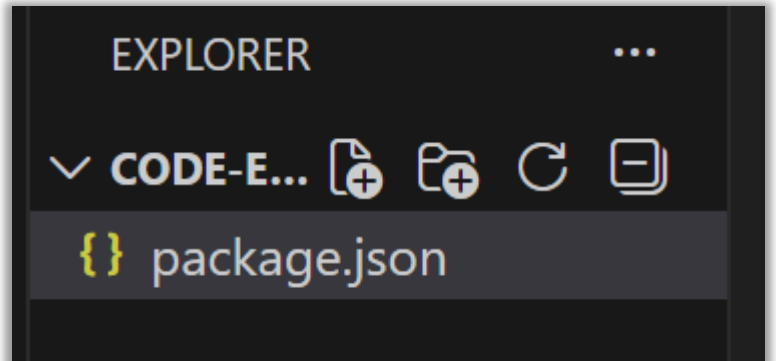
GET /data/students
GET /data/students/1
POST /data/students/
PUT /data/students/2
DELETE /data/students/3

Web Service to Manage Students

- Create a folder 'code-examples' and open in VS Code
- Open terminal and run the command

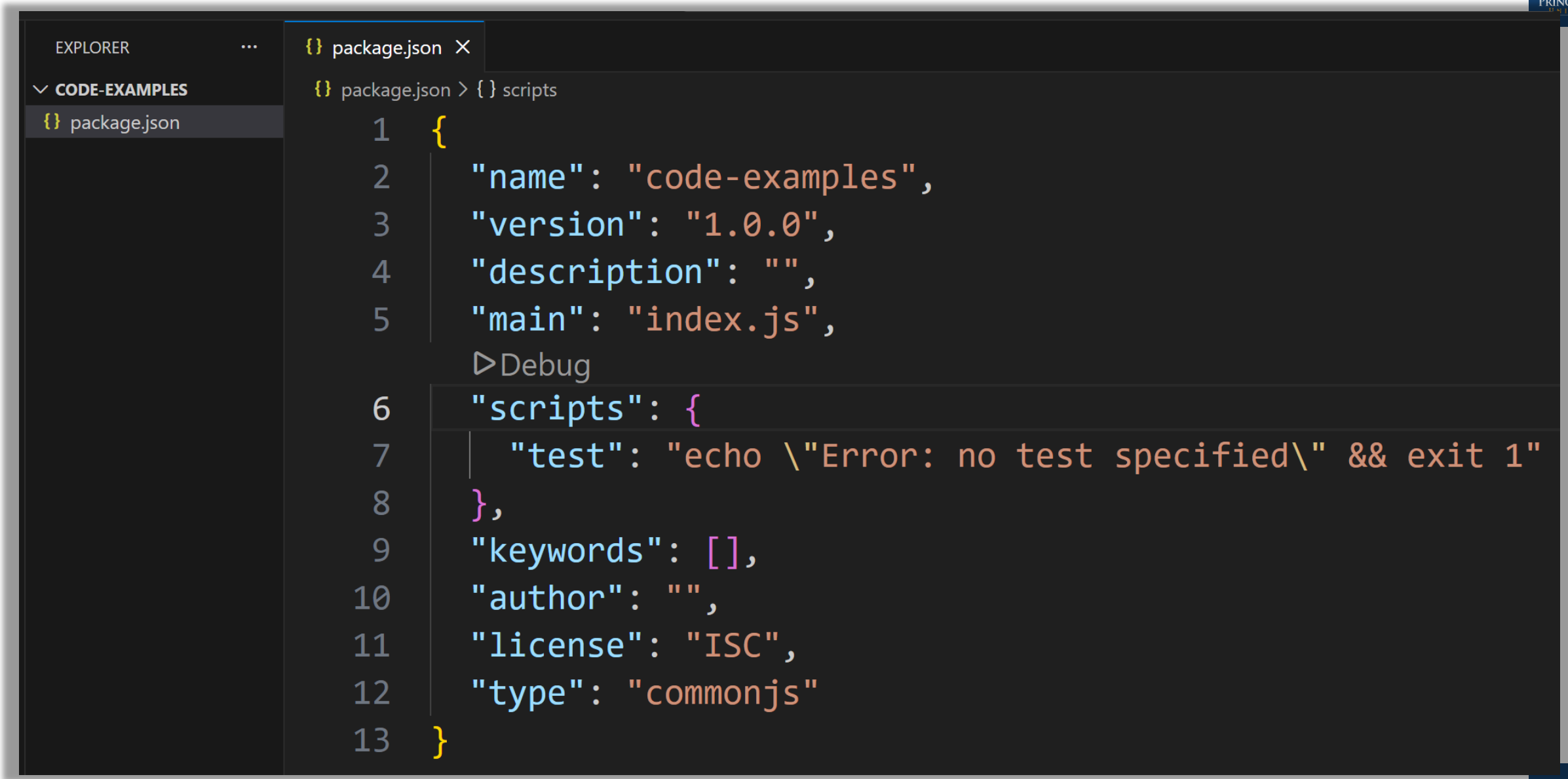
```
npm init -y
```

- It will create a package.json file in this folder



package.json

```
$ npm init -y
Wrote to D:\Teaching\IS311 Web Development\Lectures\Week 09\Code\code-examples\package.json:

{
  "name": "code-examples",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

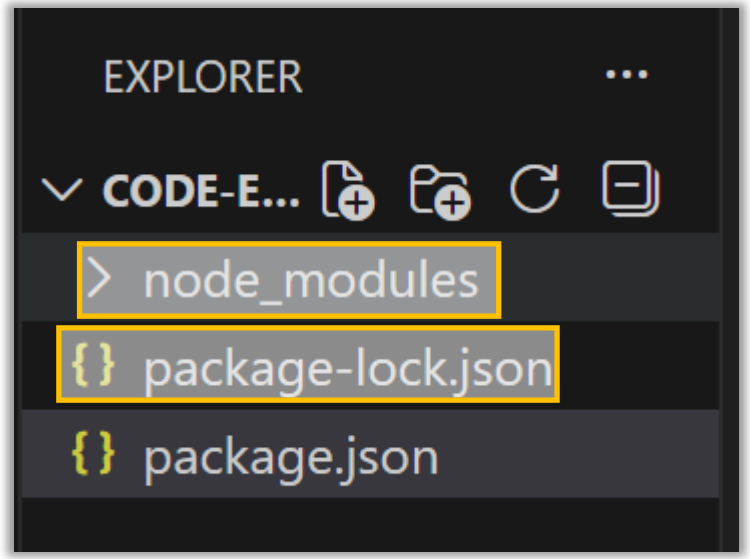

EXPLORER
CODE-EXAMPLES
package.json
package.json
package.json > { } scripts
{
"name": "code-examples",
"version": "1.0.0",
"description": "",
"main": "index.js",
Debug
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "commonjs"
}

Installing Express

- In terminal, run the following command

```
npm i express
```

- This will add a few new files in this folder

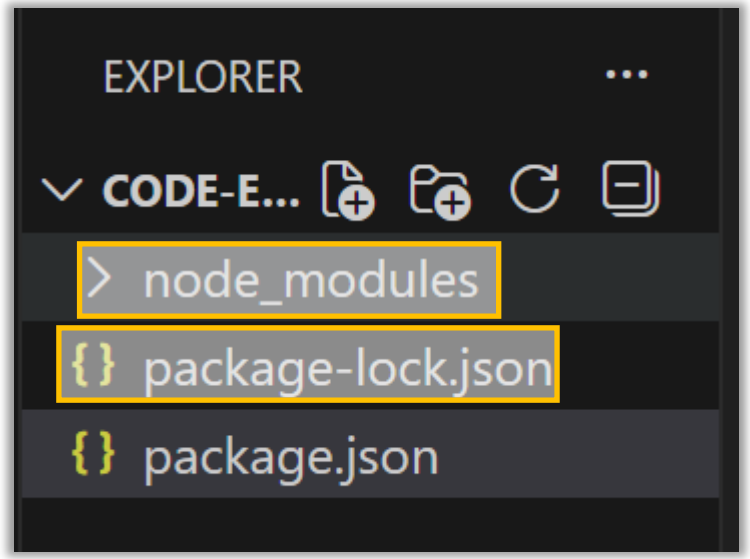


Installing nodemon

- In terminal, run the following command

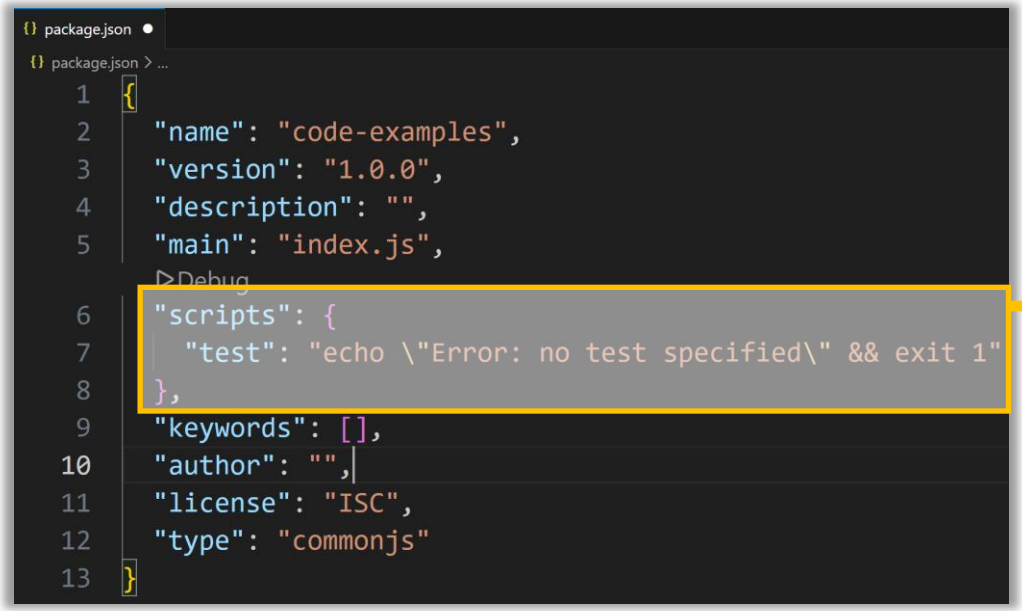
  npm i –D nodemon

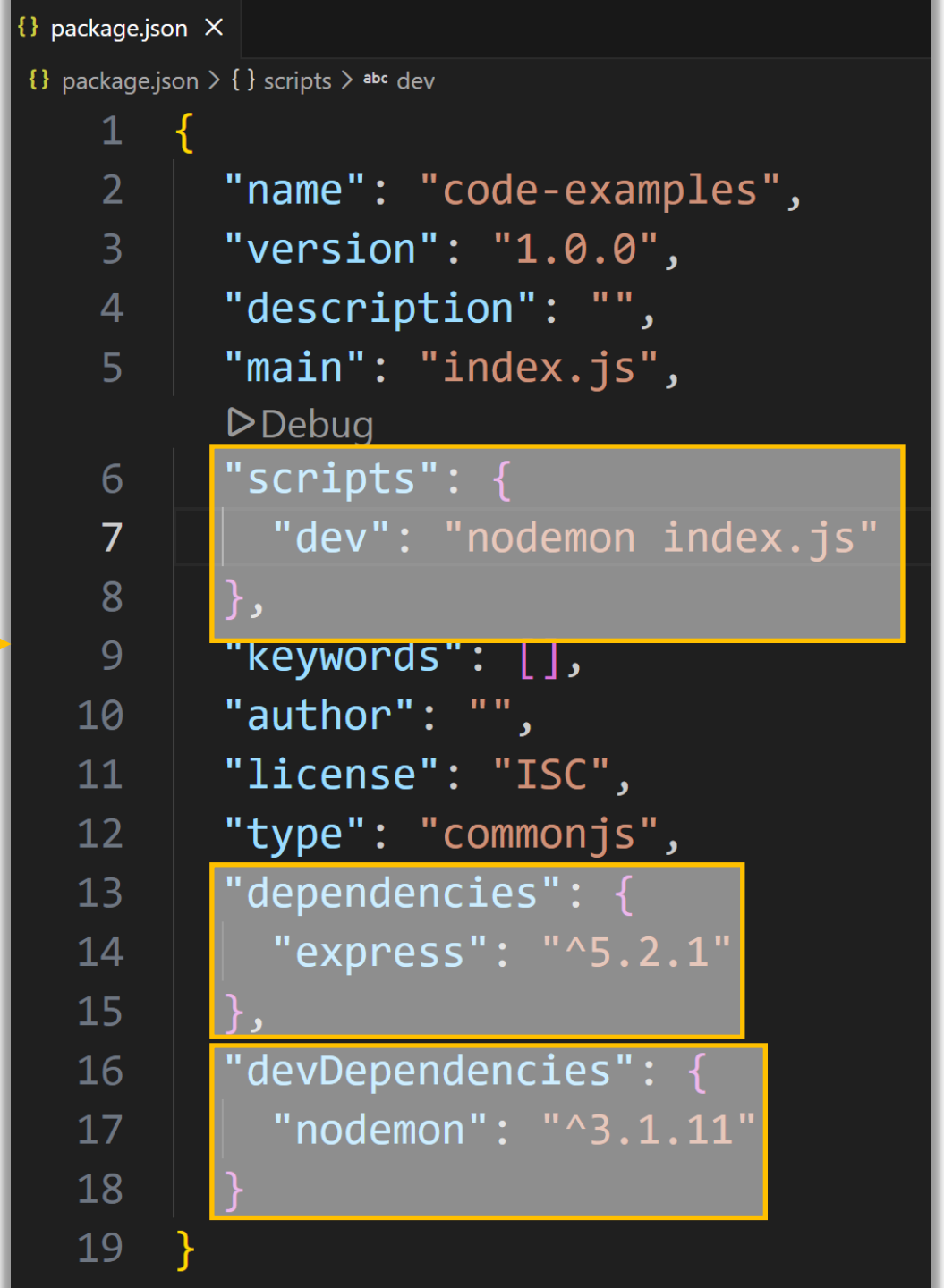
- This will add a few new files in this folder



package.json

Initial file





Express

- API Documentation
  - https://expressjs.com/en/4x/api.html
- Application
- Request
- Response
- Router

index.js

- In VS Code, create a new file and name it index.js
- Import express module and edit the file

index.js

```
const express = require("express");
```

```
const app = express();
```

```
express(): Express
```

Creates an Express application. The express() function is a top-level function exported by the express module.

index.js

```
const express = require("express");
const app = express();
```

```
app.get("URL", function (req, res) {});
app.post("URL", function (req, res) {});
app.put("URL", function (req, res) {});
app.delete("URL", function (req, res) {});
```

These functions can take two arguments:
(1) String route
(2) Callback function

index.js

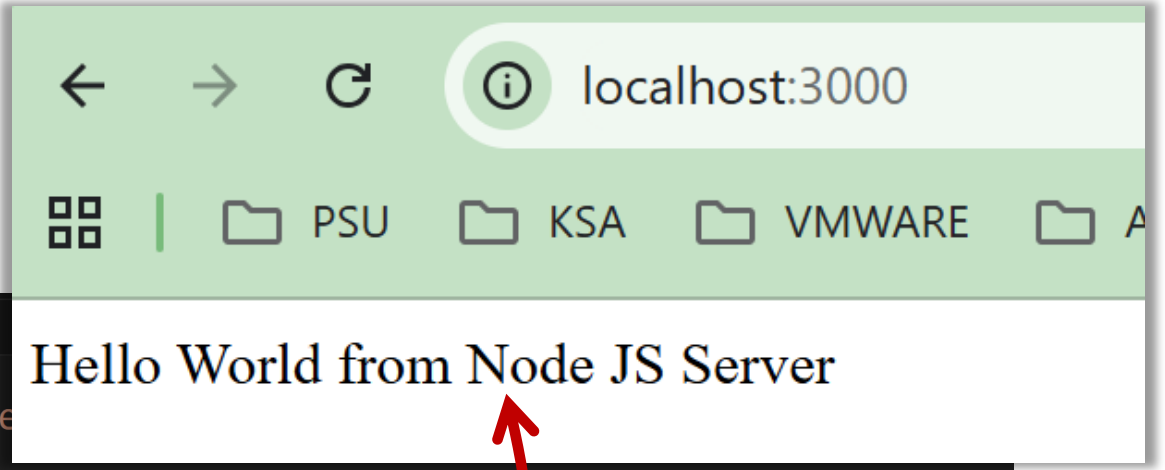
```
const express = require("express");
const app = express();
console.log("Express App is created...");

app.get("/", (req, res) => {
  res.send("Hello World from Node JS Server");
});
```

Implemented a route "/" to root that will return the 'Hello World!!!' message to the browser

index.js

```
const express = require("express");
const app = express();
console.log("Express App is created...");

app.get("/", (req, res) => {
  res.send("Hello World from Node JS Server");
});

app.listen(3000, () =>
  console.log("Express App is waiting on port 3000 for any HTTP
request!!!")
);
```

index.js




```
EXPLORER
CODE-EXAMPLES
  node_modules
  index.js
  package-lock.json
  package.json
```

index.js

```
1  const express = require("expre
2  const app = express();
3
4  console.log("Express App is created...");
5
6  app.get("/", (req, res) => {
7    res.send("Hello World from Node JS Server");
8  });
9
10 app.listen(3000, () =>
11   console.log("Express App is waiting on port 3000 for any HTTP request!!!")
12 );
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
malik@DESKTOP-HVT445C MINGW64 /d/Teaching/IS311 Web Development/Lectures/Week 09/Code/code-examples
$ node index.js
Express App is created...
Express App is waiting on port 3000 for any HTTP request!!!
```

node index.js

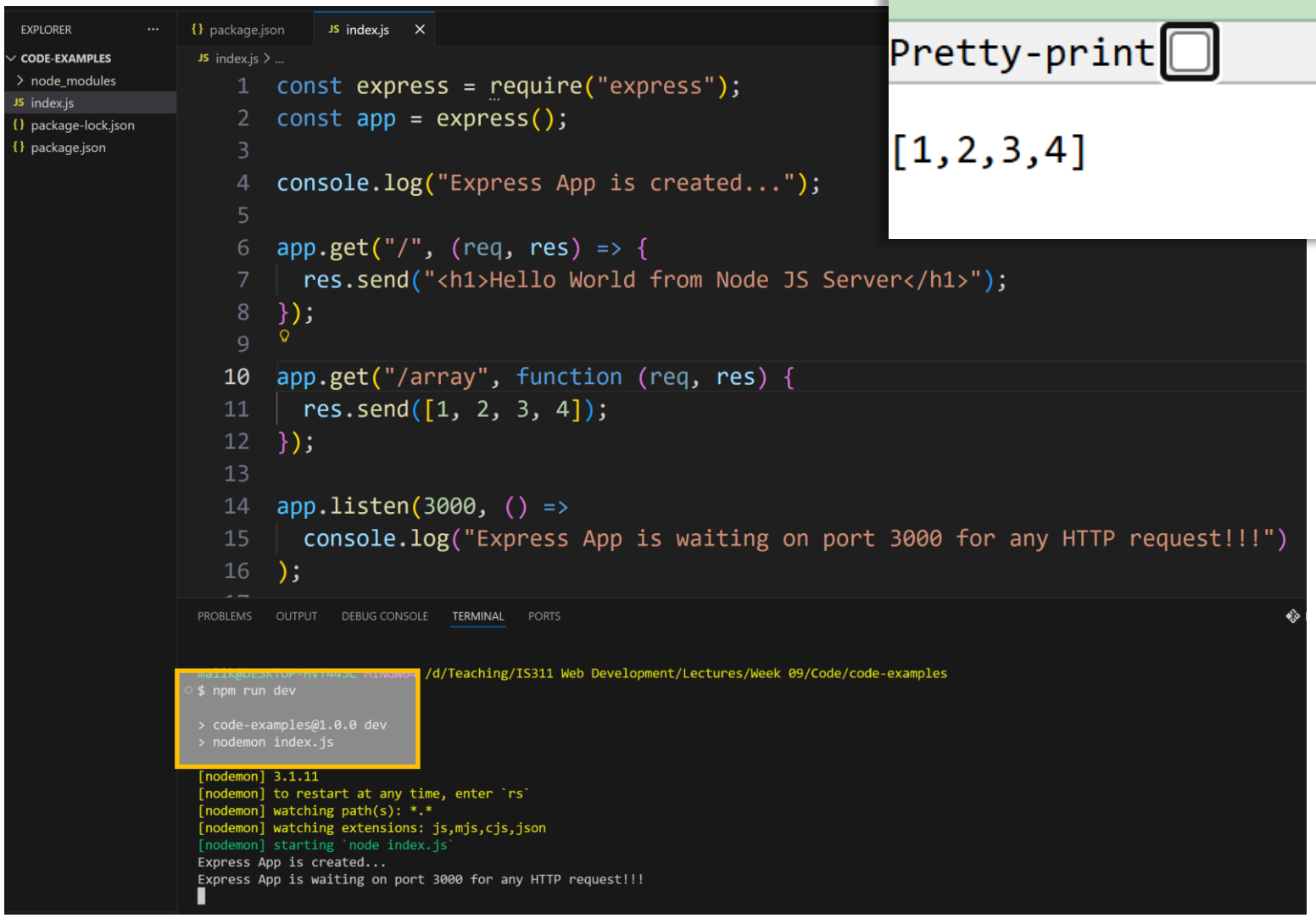
index.js

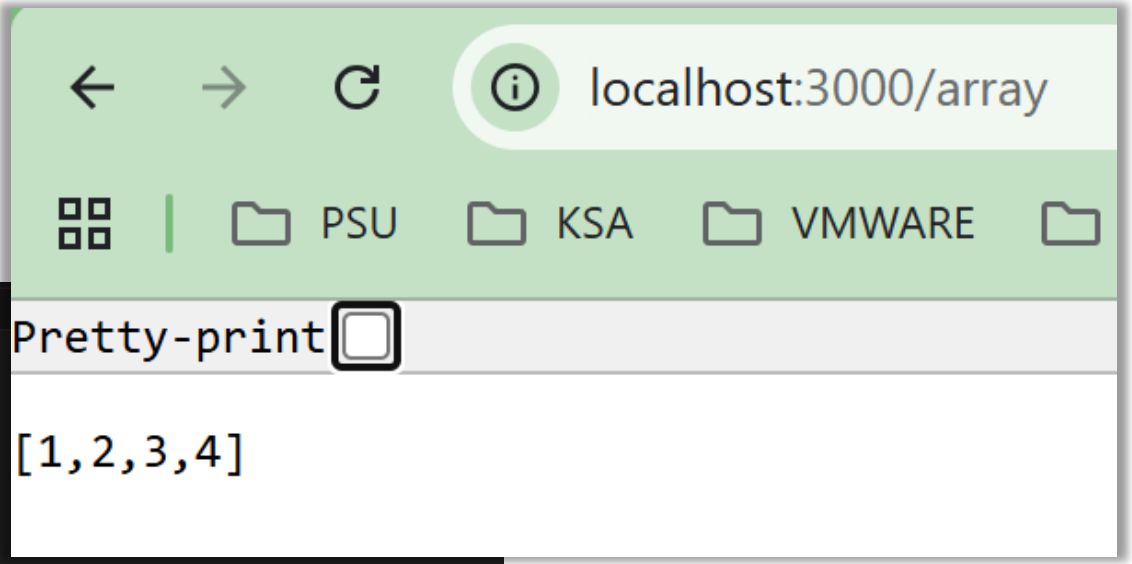
- Add another route in this file with GET method

```
app.get("/array", function (req, res) {
  res.send([1, 2, 3, 4]);
});
```

Stope the previous execution by pressing "ctrl + c" and run again the newly modified program after saving index.js file

index.js





npm run dev

nodemon Package

- To get rid of running program again and again, we can install nodemon package using NPM
- To install run the followning command

```
npm i -D nodemon
```

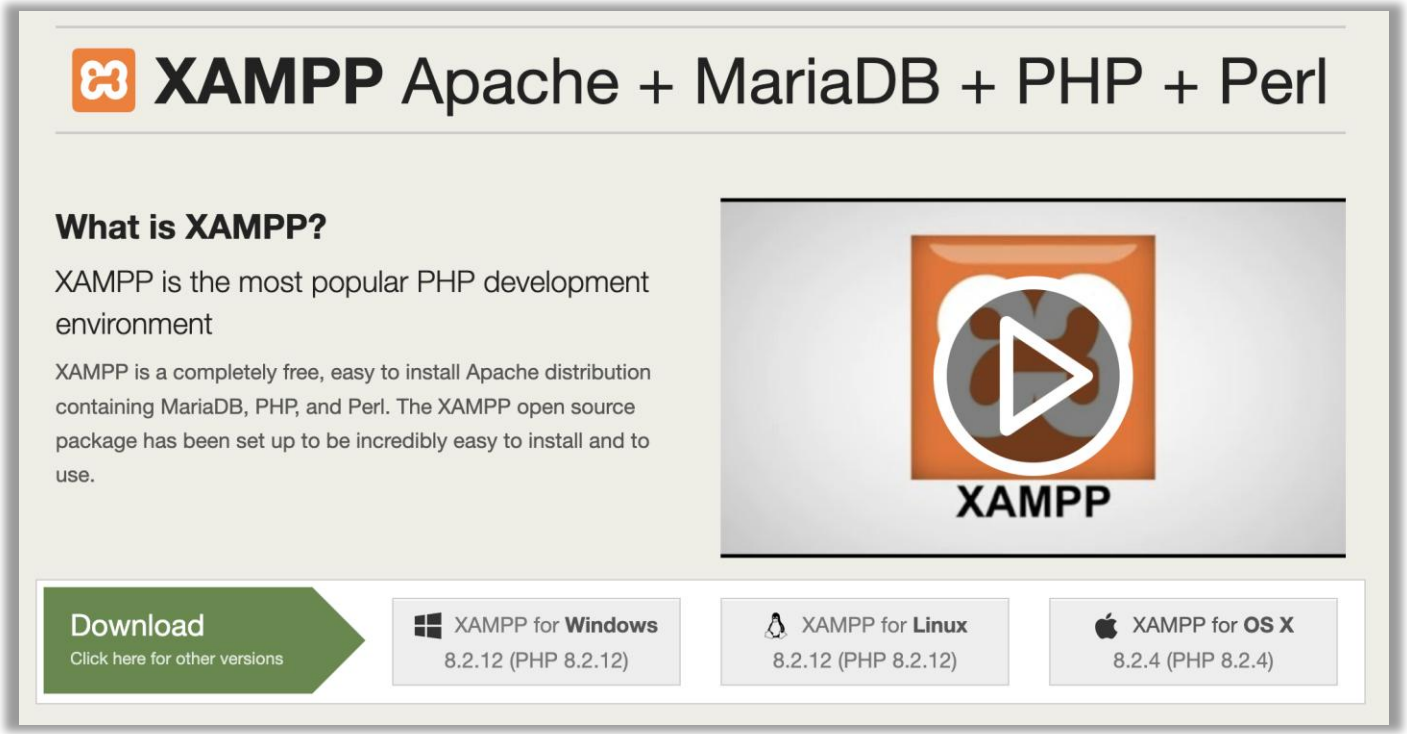
- Edit package.json file and run the server with nodemon

Dynamic Web Development

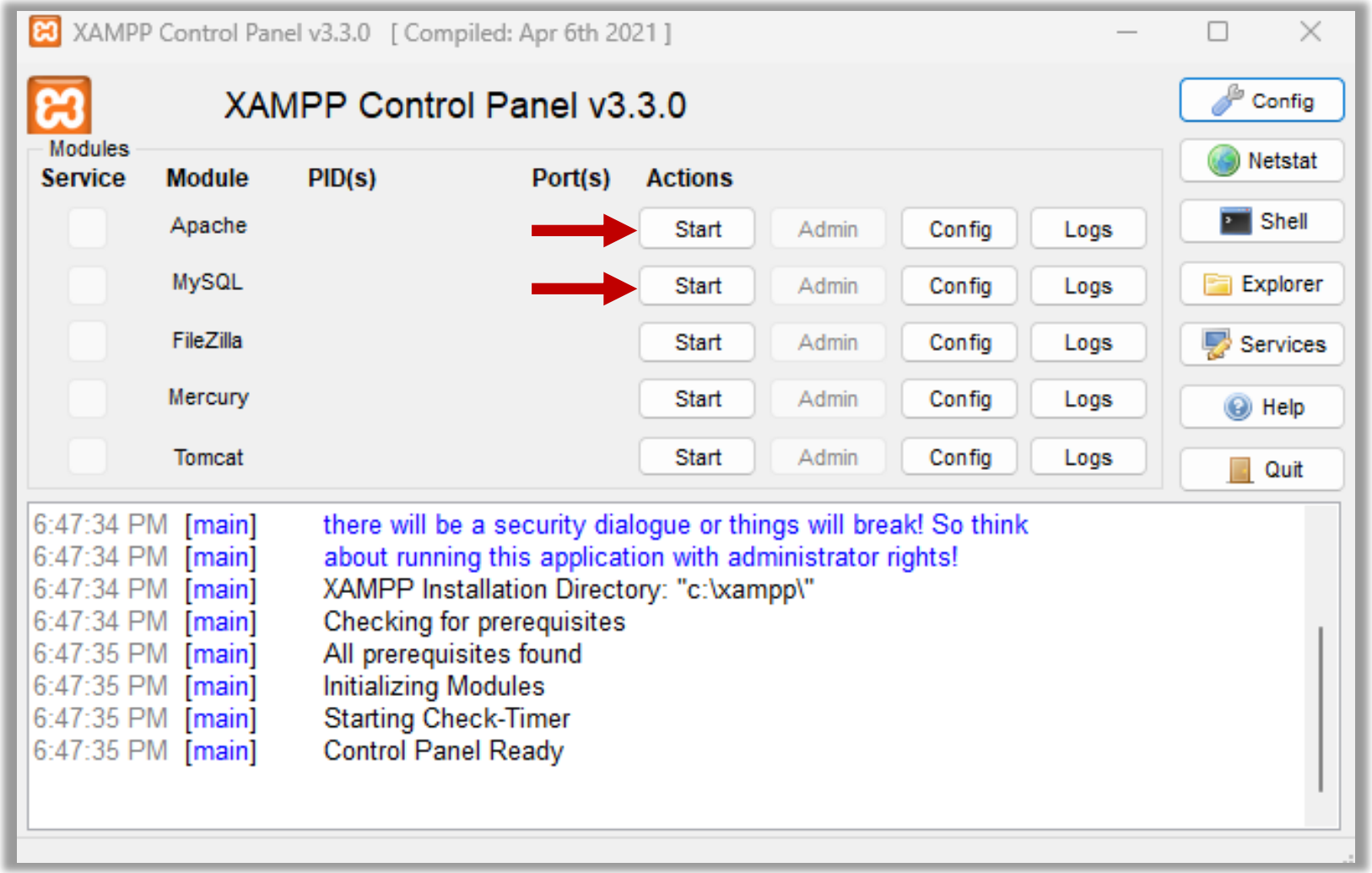
- Data is coming from different soruces
  - Databse
  - Cloud
  - Files
  - Streams
  - …
- Based on data, web pages are changing – making them dynamic

MySQL DBMS

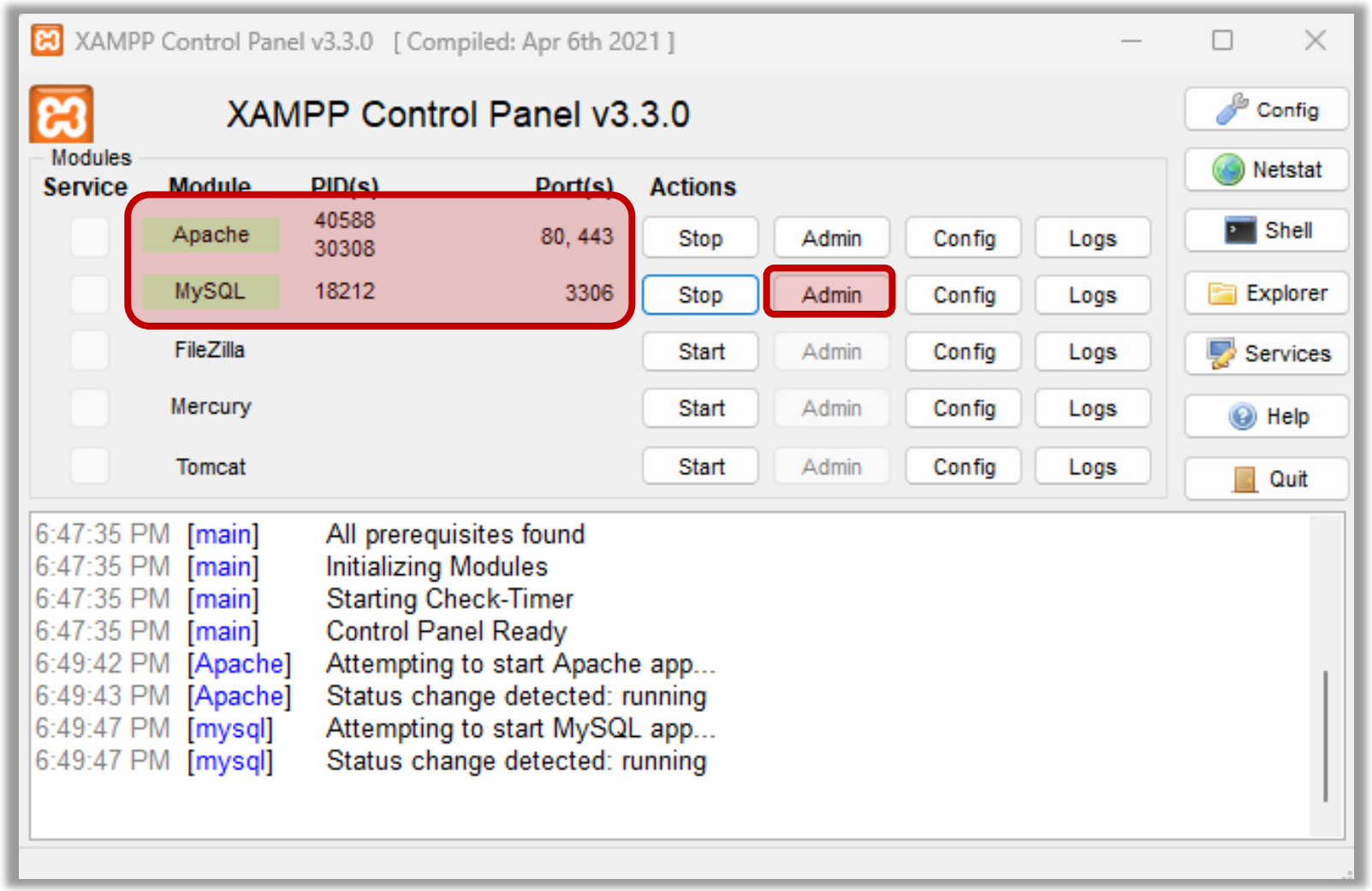
- To install MySQL
  - https://www.apachefriends.org/index.html

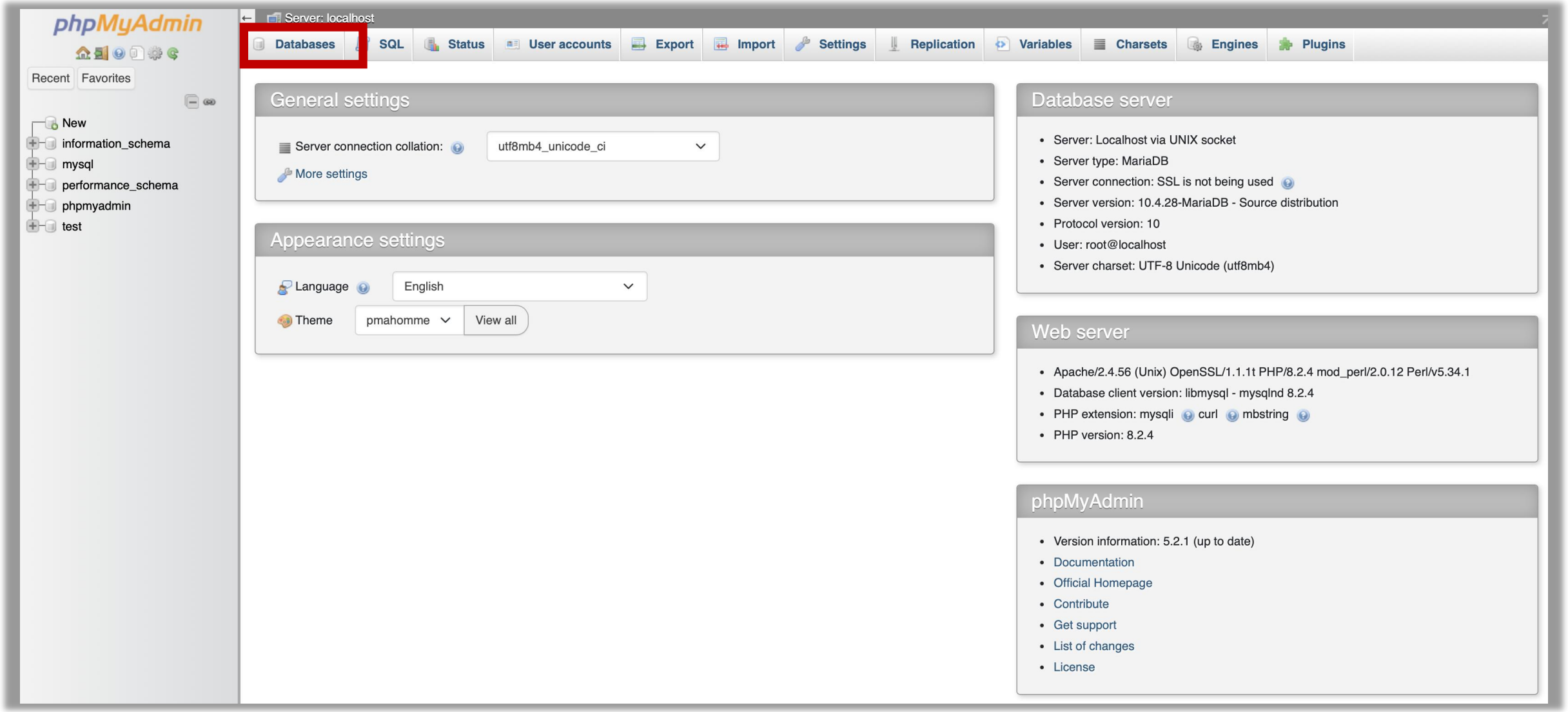


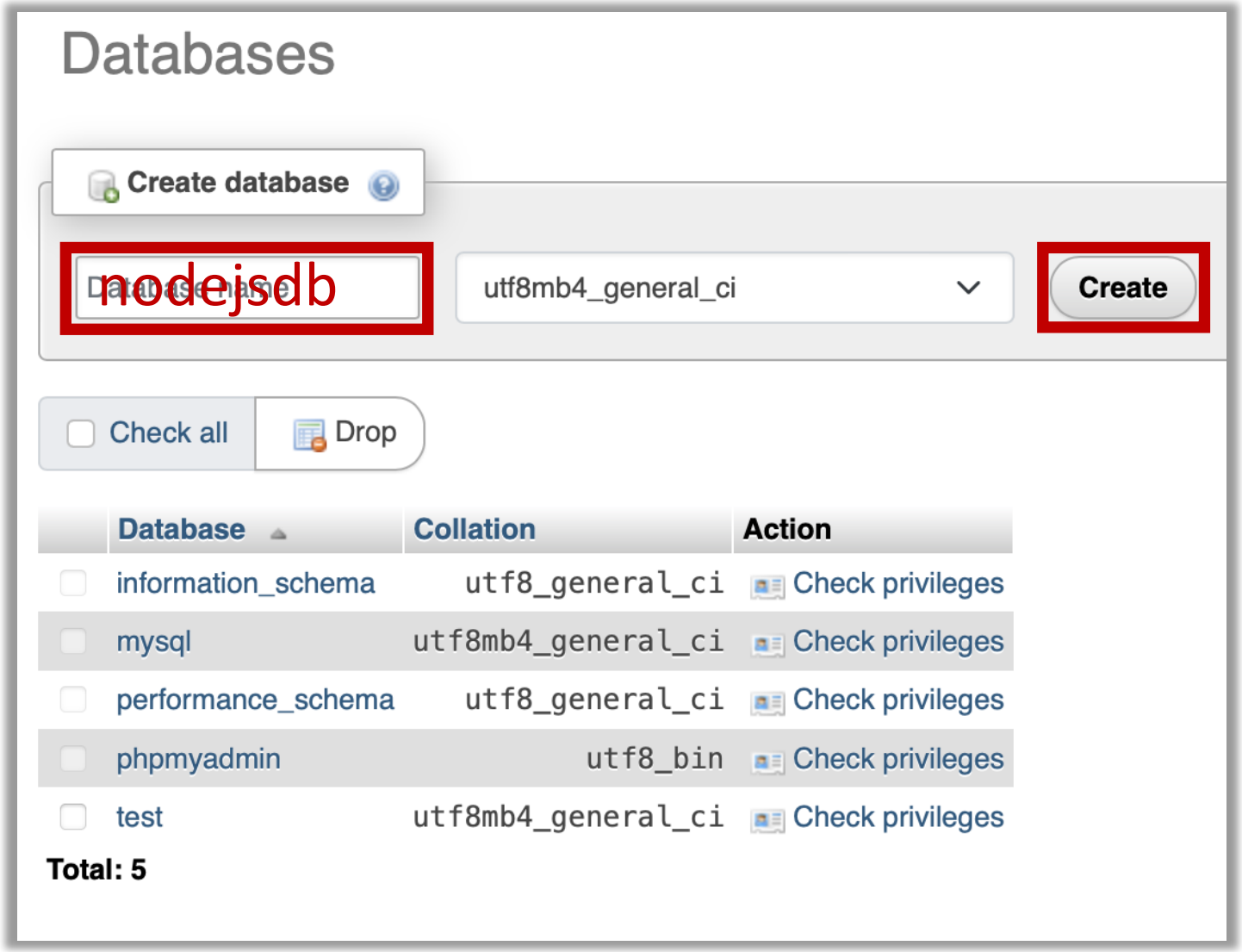
XAMPP - Admin

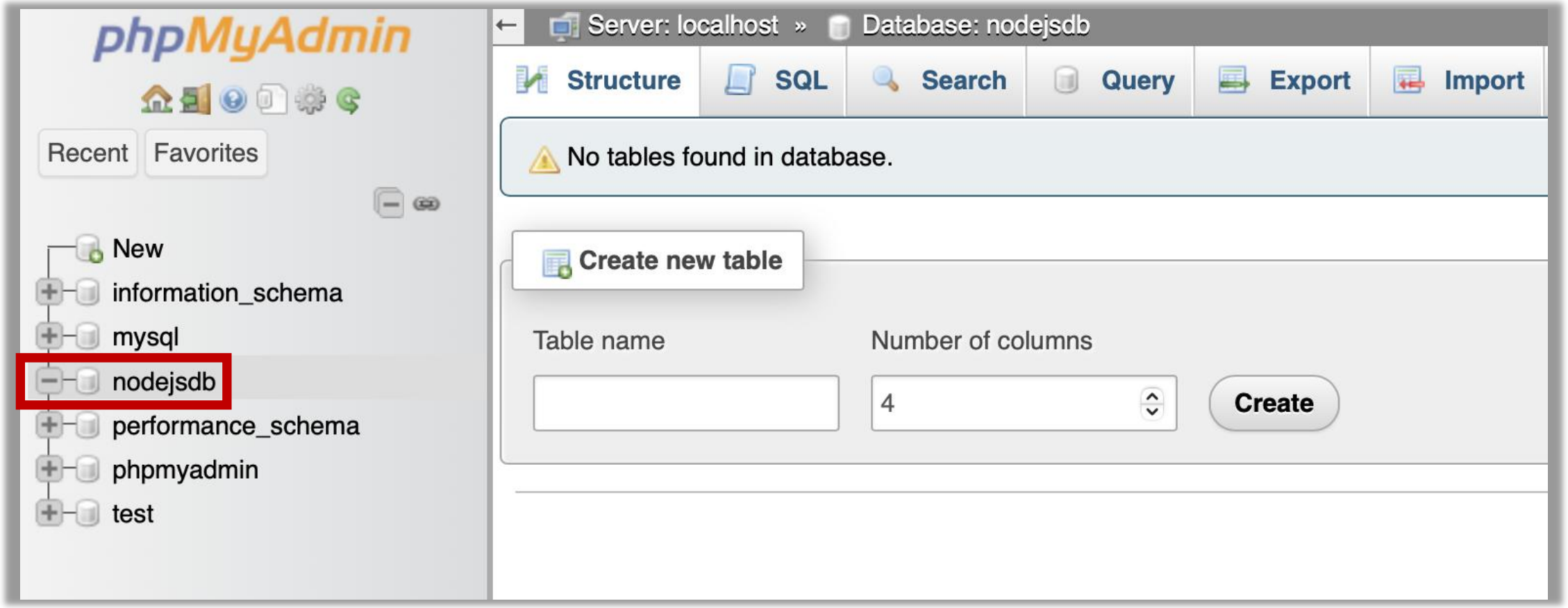

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]
XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
MySQL
FileZilla
Mercury
Tomcat
Start
Admin
Config
Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
6:47:34 PM [main] there will be a security dialogue or things will break! So think
6:47:34 PM [main] about running this application with administrator rights!
6:47:34 PM [main] XAMPP Installation Directory: "c:\xampp\"
6:47:34 PM [main] Checking for prerequisites
6:47:35 PM [main] All prerequisites found
6:47:35 PM [main] Initializing Modules
6:47:35 PM [main] Starting Check-Timer
6:47:35 PM [main] Control Panel Ready

XAMPP - Admin


XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]
XAMPP Control Panel v3.3.0
Modules
Service
Module
PID(s)
Port(s)
Actions
Apache
40588
30308
80, 443
Stop
Admin
Config
Logs
MySQL
18212
3306
Stop
Admin
Config
Logs
FileZilla
Start
Admin
Config
Logs
Mercury
Start
Admin
Config
Logs
Tomcat
Start
Admin
Config
Logs
Config
Netstat
Shell
Explorer
Services
Help
Quit
6:47:35 PM [main] All prerequisites found
6:47:35 PM [main] Initializing Modules
6:47:35 PM [main] Starting Check-Timer
6:47:35 PM [main] Control Panel Ready
6:49:42 PM [Apache] Attempting to start Apache app...
6:49:43 PM [Apache] Status change detected: running
6:49:47 PM [mysql] Attempting to start MySQL app...
6:49:47 PM [mysql] Status change detected: running


phpMyAdmin
Recent
Favorites
New
information_schema
mysql
performance_schema
phpmyadmin
test
Server: localhost
Databases
SQL
Status
User accounts
Export
Import
Settings
Replication
Variables
Charsets
Engines
Plugins
General settings
Server connection collation:
utf8mb4_unicode_ci
More settings
Appearance settings
Language
English
Theme
pmahomme
View all
Database server
Server: Localhost via UNIX socket
Server type: MariaDB
Server connection: SSL is not being used
Server version: 10.4.28-MariaDB - Source distribution
Protocol version: 10
User: root@localhost
Server charset: UTF-8 Unicode (utf8mb4)
Web server
Apache/2.4.56 (Unix) OpenSSL/1.1.1t PHP/8.2.4 mod_perl/2.0.12 Perl/v5.34.1
Database client version: libmysql - mysqlnd 8.2.4
PHP extension: mysqli curl mbstring
PHP version: 8.2.4
phpMyAdmin
Version information: 5.2.1 (up to date)
Documentation
Official Homepage
Contribute
Get support
List of changes
License


Databases
Create database
nodejsdb
utf8mb4_general_ci
Create
Check all
Drop
Database
Collation
Action
information_schema utf8_general_ci Check privileges
mysql utf8mb4_general_ci Check privileges
performance_schema utf8_general_ci Check privileges
phpmyadmin utf8_bin Check privileges
test utf8mb4_general_ci Check privileges
Total: 5

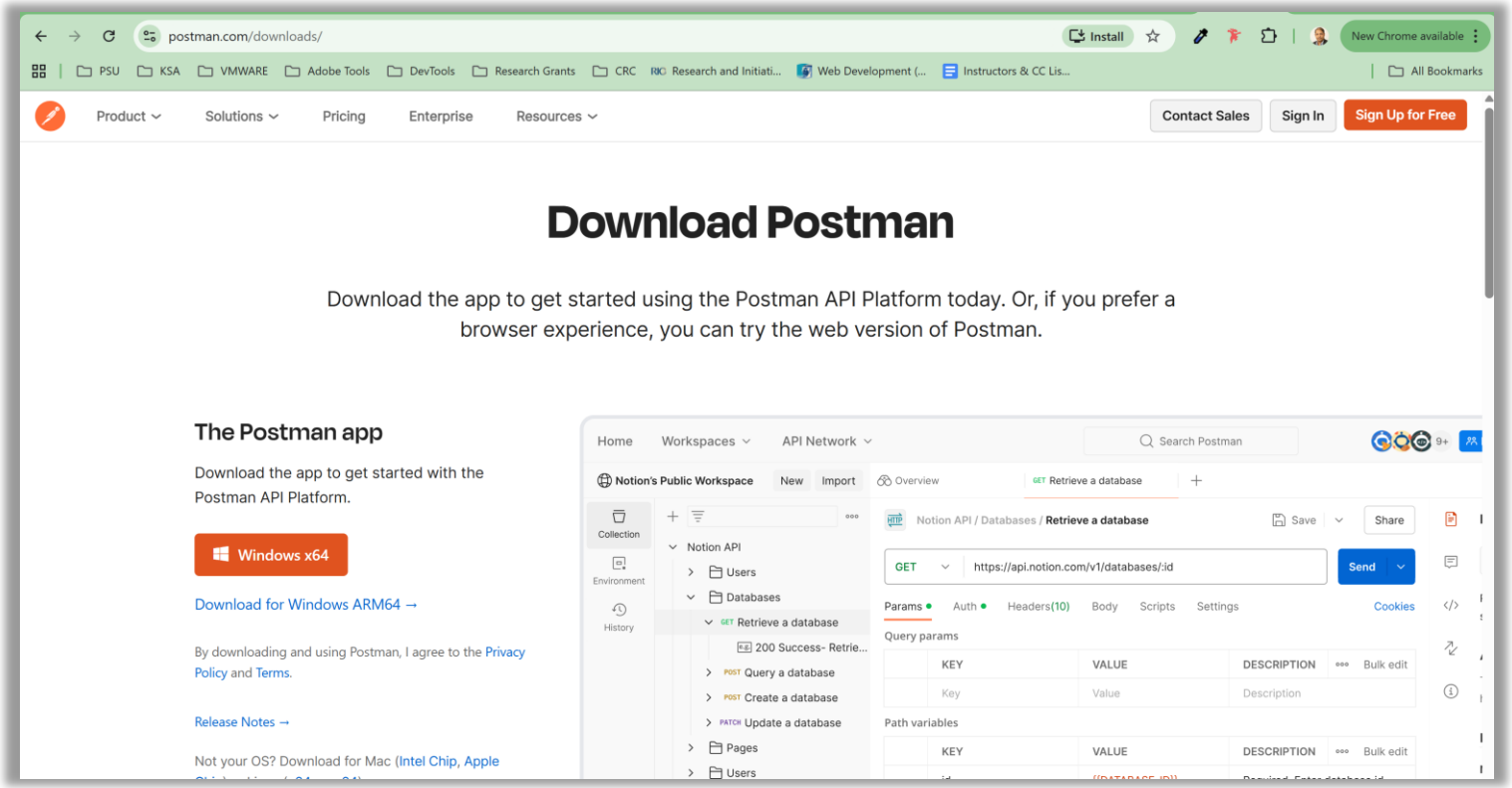

phpMyAdmin
Recent
Favorites
New
information_schema
mysql
nodejsdb
performance_schema
phpmyadmin
test
Server: localhost » Database: nodejsdb
Structure
SQL
Search
Query
Export
Import
No tables found in database.
Create new table
Table name
Number of columns
4
Create

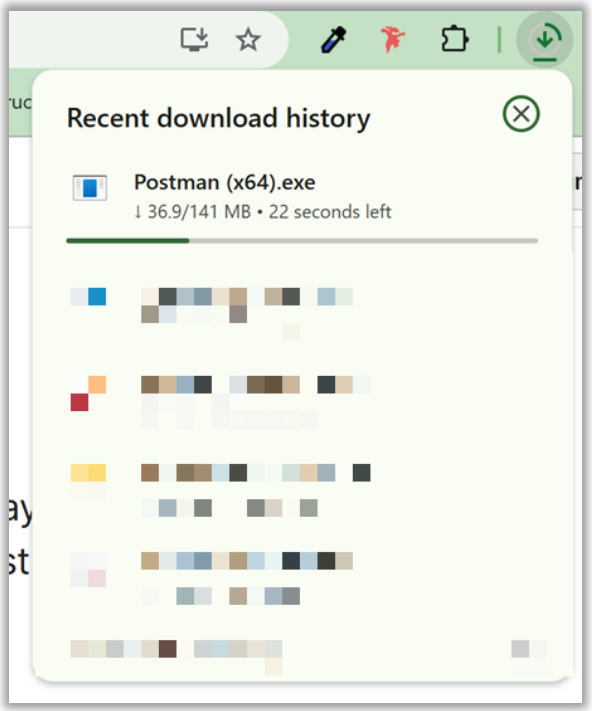
MySQL DBMS

- To learn how to code in SLQ in MYSQL
- Please follow the following tutorial
- https://www.w3schools.com/mysql/default.asp

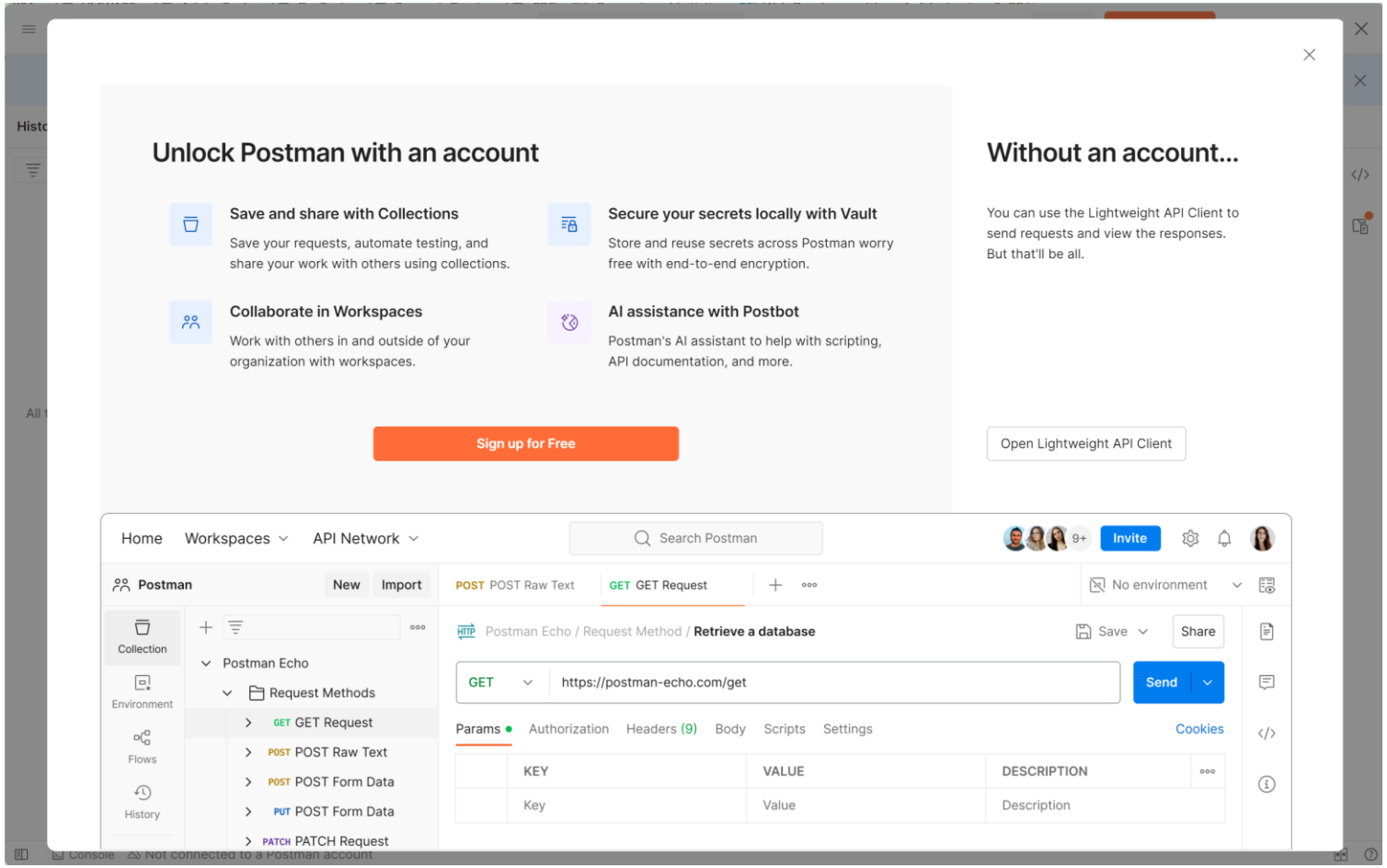
Postman Application

- https://www.postman.com/downloads/

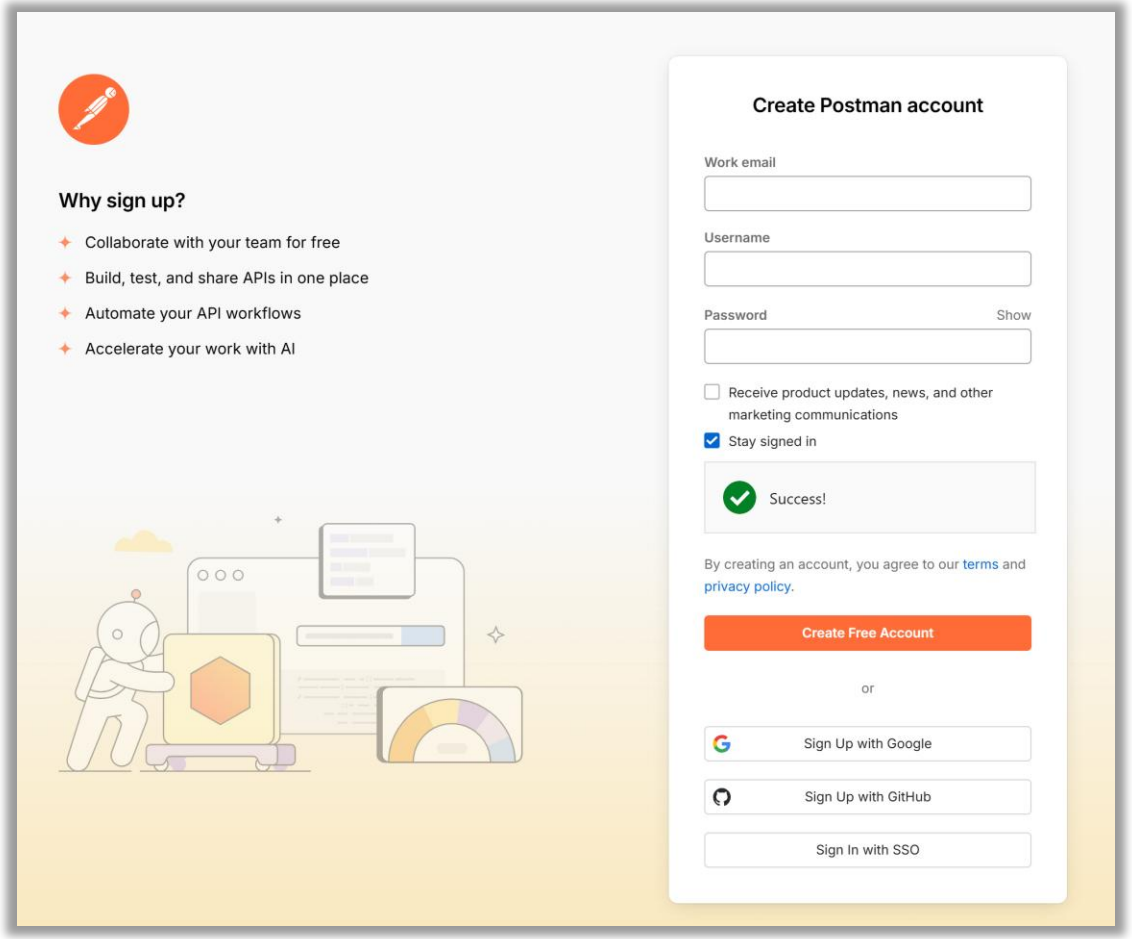




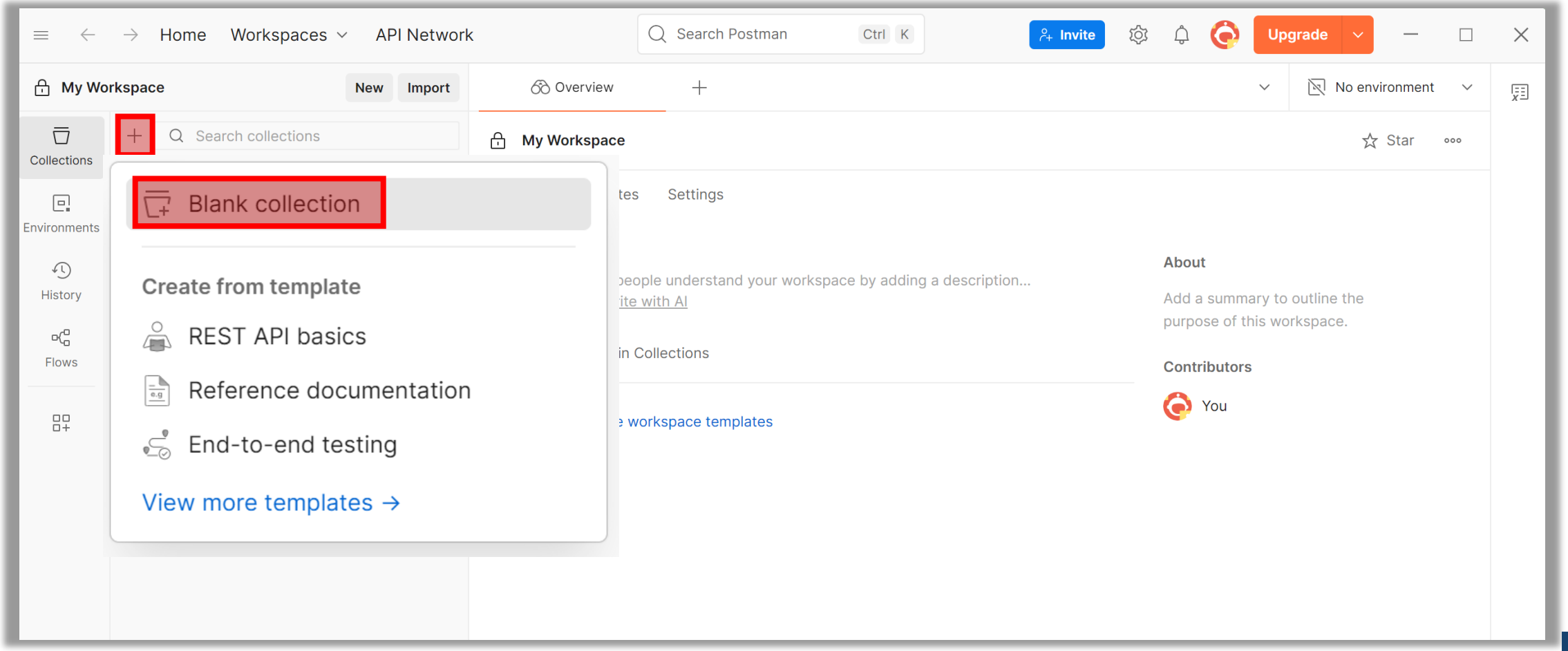
Postman Application



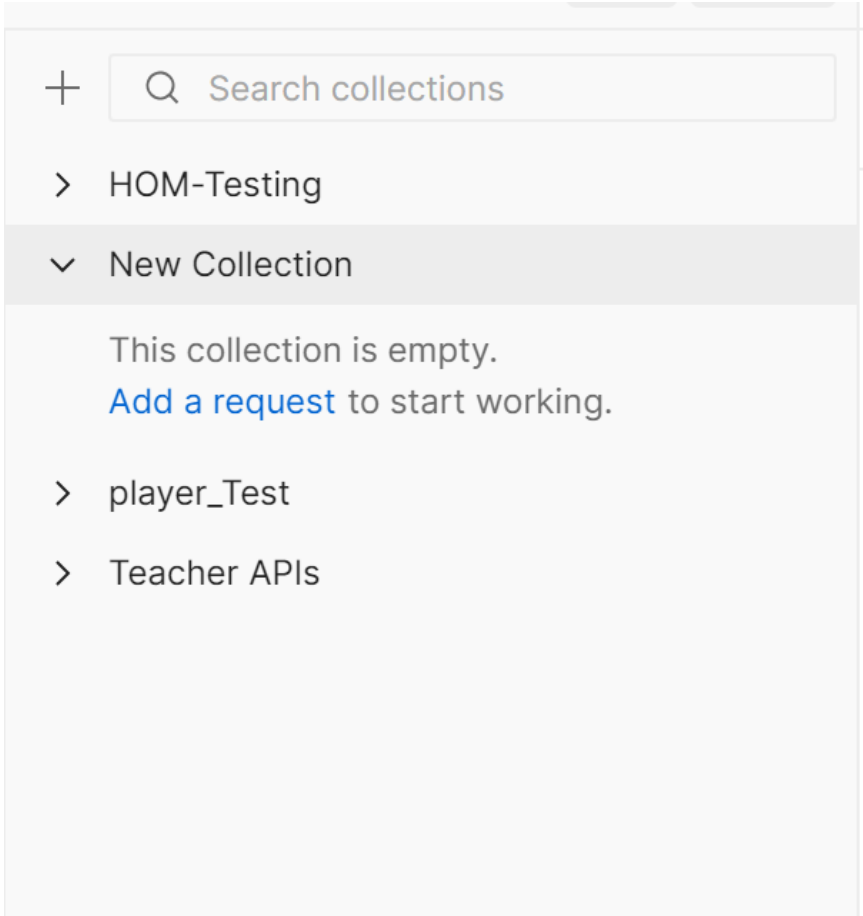
Postman Application

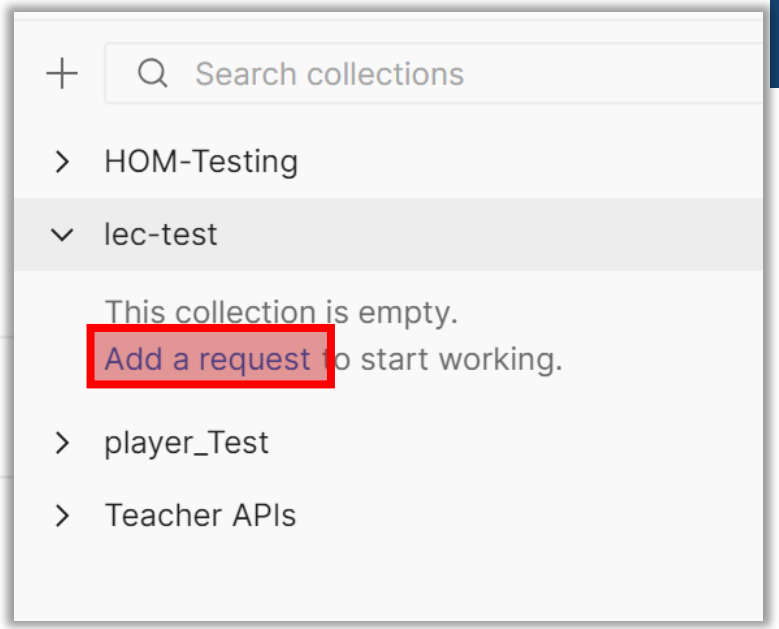


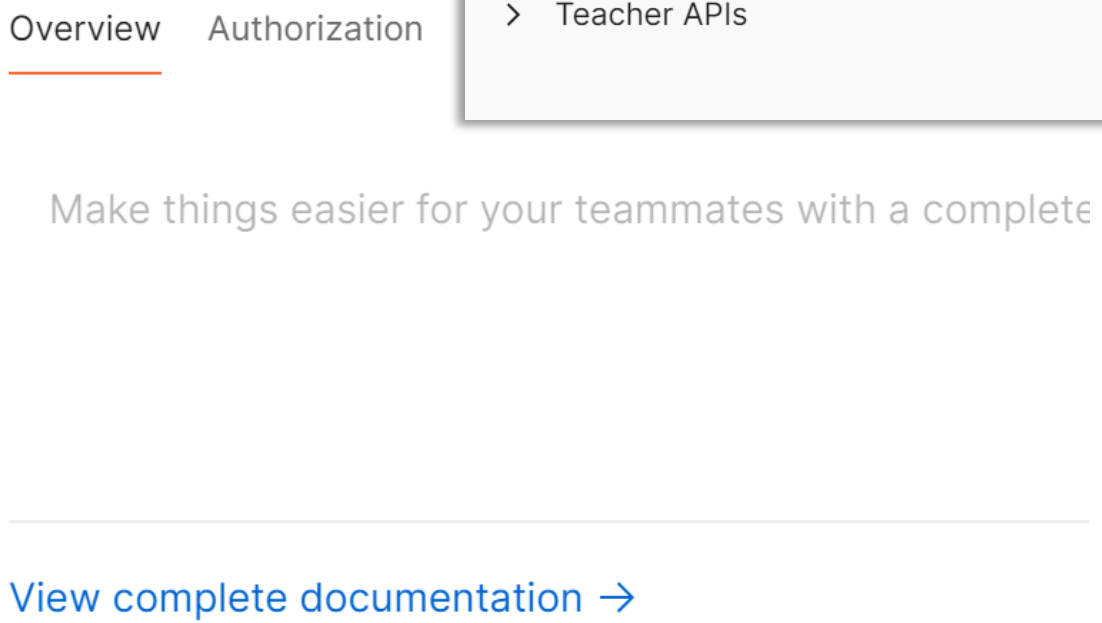
Postman Application



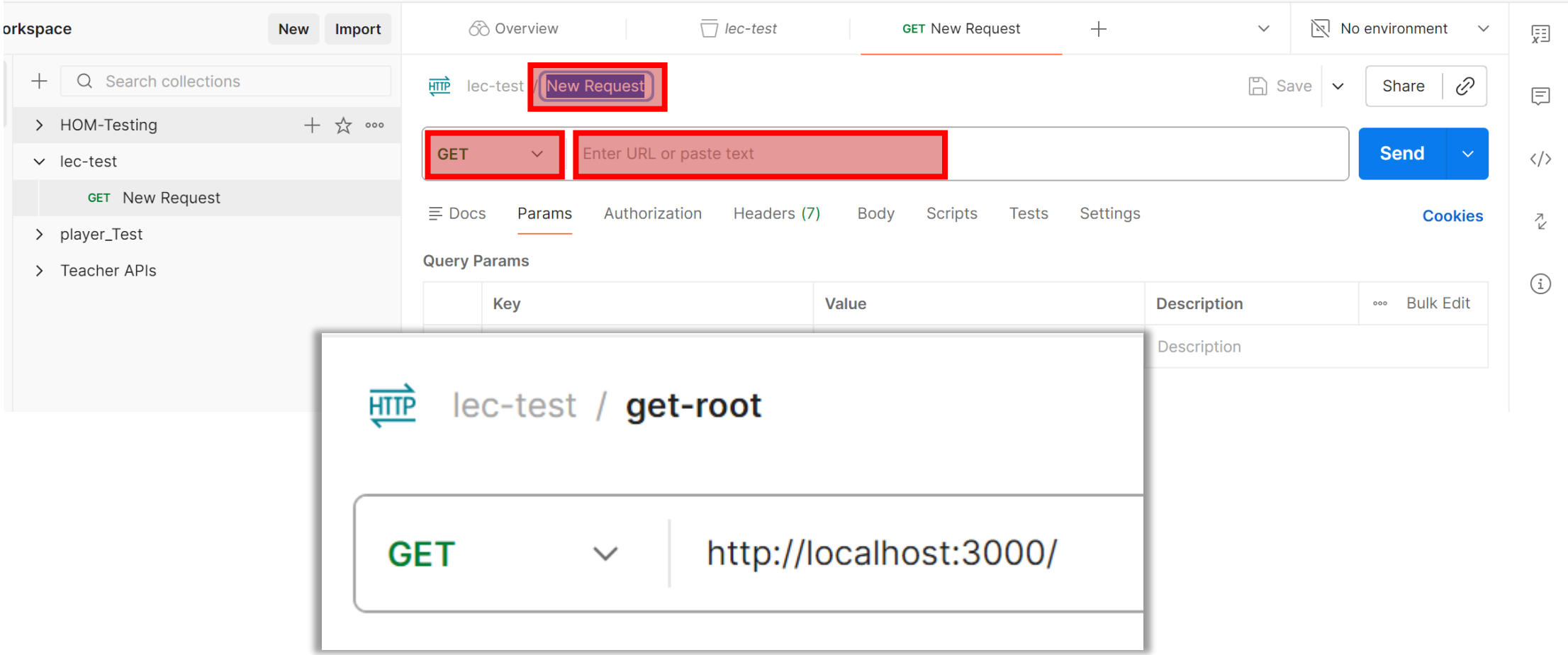
Postman Application



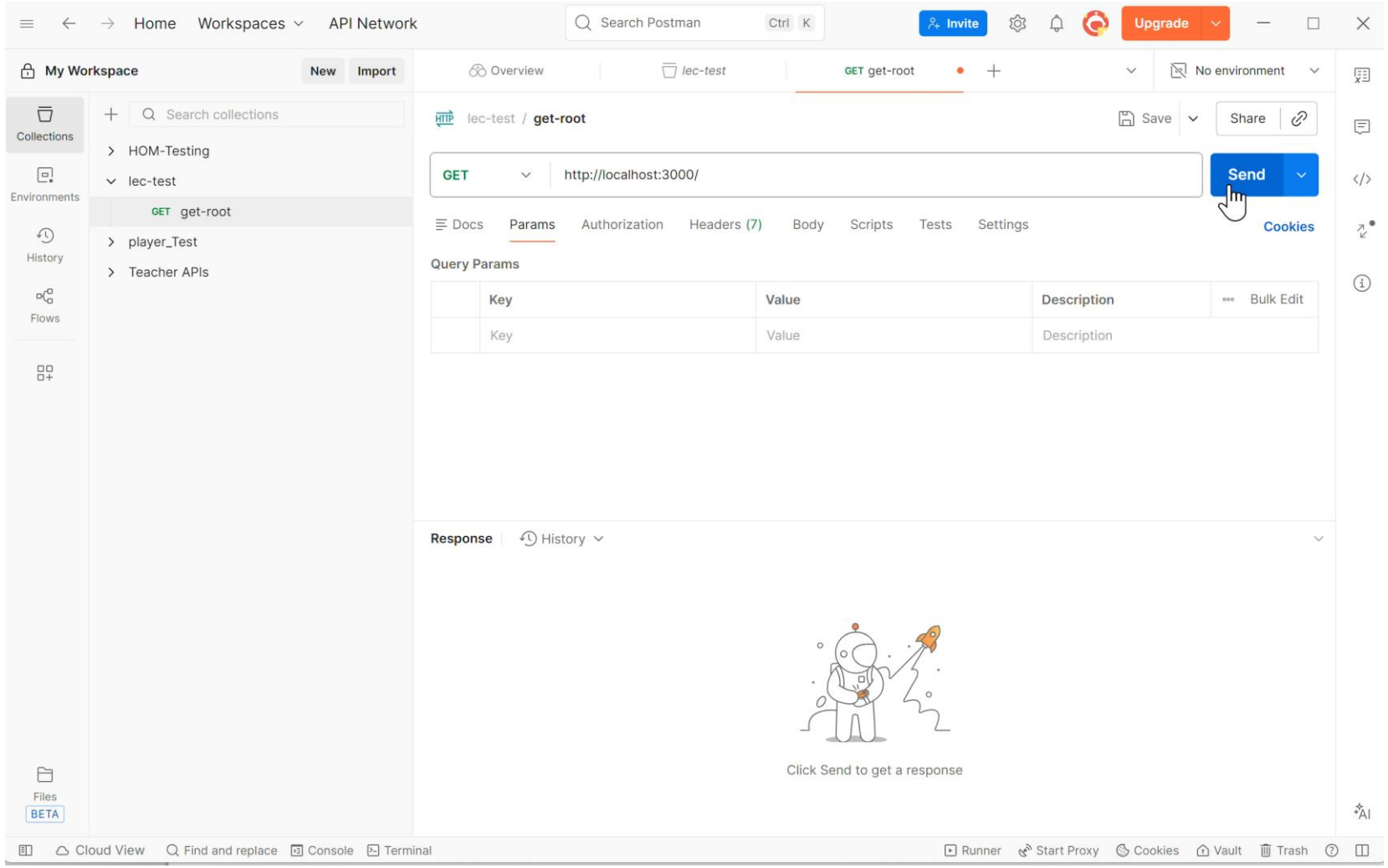




Postman Application



Postman Application



Database Connetions with Node.JS

- Create a new folder and name it 'dbwebservice'
- Open this folder in VS Code
- Open terminal or command prompt in VS Code
- Run the command

```
npm init -yes
```

Configure the Project

- In VS Code, open terminal:
  - Terminal -> New Terminal
- Run the following commands in this terminal to install express and mysql packages

```
npm i express
npm i mysql2
```

→ `npm i express mysql2`

```
npm i -D nodemon
```

MYSQL Documentation: https://www.npmjs.com/package/mysql

package.json

Initial file

package.json

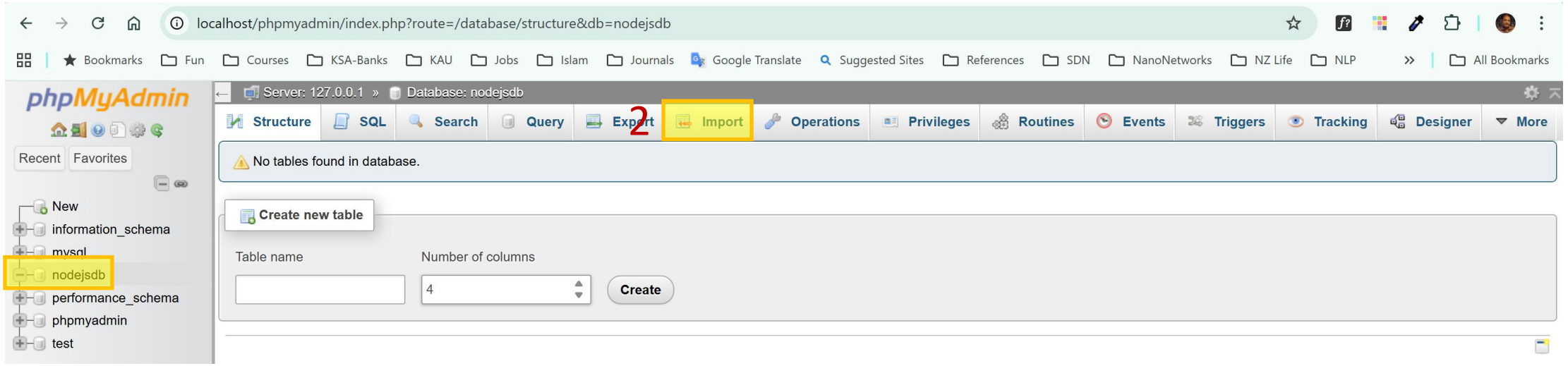
```
{
  "name": "code-examples",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
{
  "name": "dbwebservice",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "dev": "nodemon index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1",
    "mysql2": "^3.20.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

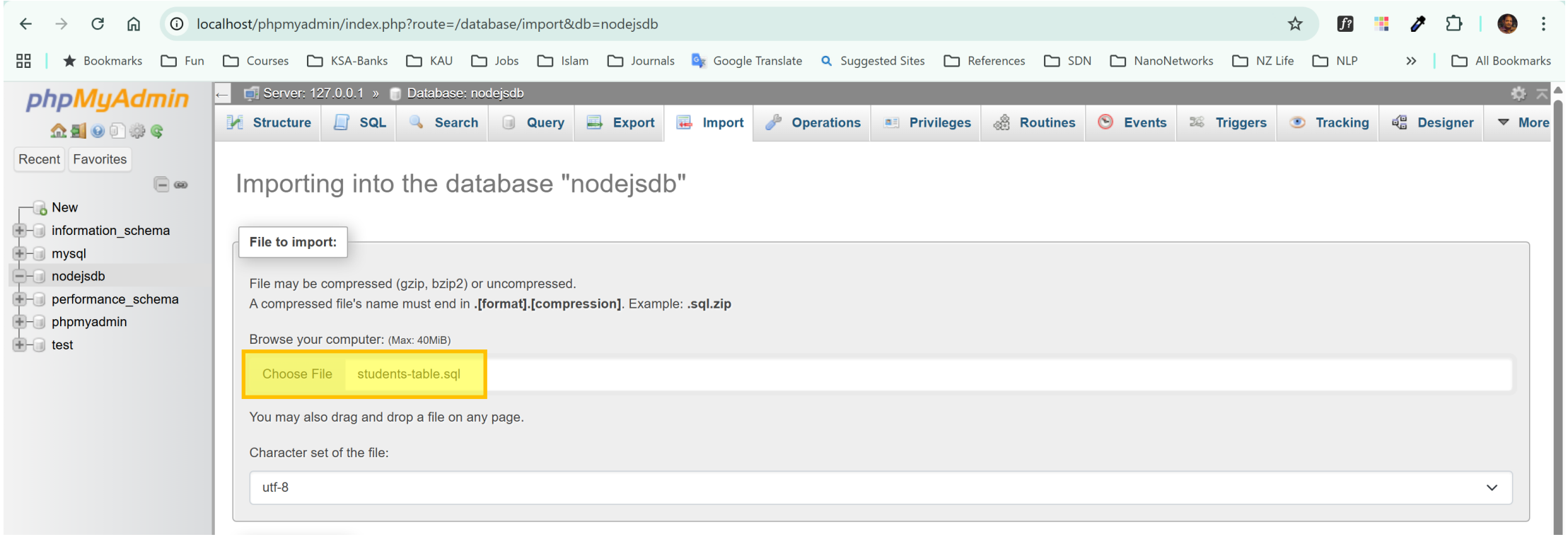
Database in XAMPP

- Open XAMPP
- Open Admin for MySQL
- Open **nodejsdb** that we have created earlier
- Import file **students-table.sql** file (download it from LMS or GitHub)
  - It creates a students table and add 30 records

Import SQL File



Import SQL File



Structure of students Table

```sql
CREATE TABLE IF NOT EXISTS students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    gender VARCHAR(10),
    semester INT NOT NULL,
    major ENUM('CS', 'IS', 'SE') NOT NULL,
    -- Add constraints
    CONSTRAINT chk_semester CHECK (semester BETWEEN 1 AND 8)
);
```

index.js

- Create a new file in the folder and name it index.js
- Import exprss and mysql packages

```
const express = require("express");
const mysql = require("mysql2");
const app = express();
```

Db Connection

- Create the database connection using mysql

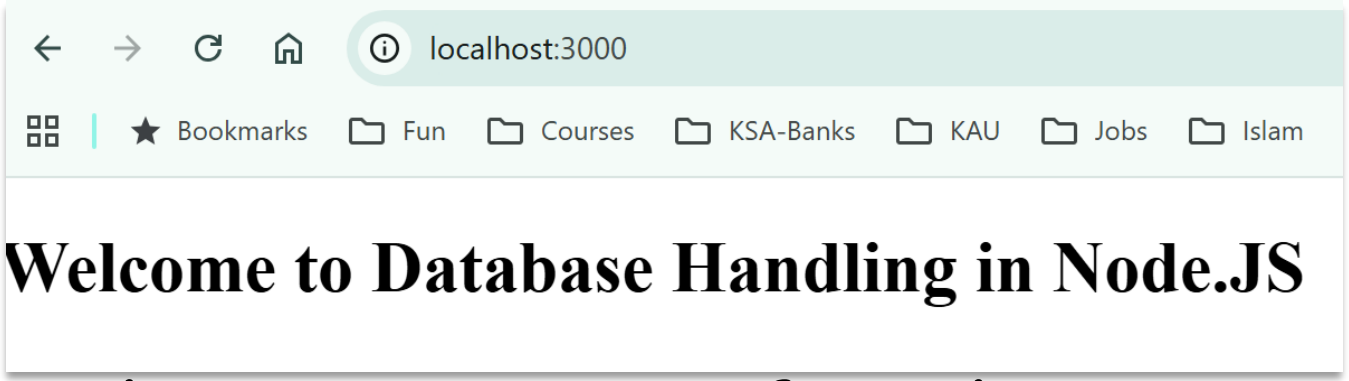
```
const dbCon = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "nodejsdb",
});
```

Db Connection

- Check the database connection

```
dbCon.connect((error) => {
  if (error) {
    console.log("Error Db: " + error.message);
    throw error;
  } else {
    console.log("Db is connected successfully...");
  }
});
```

'/' Route



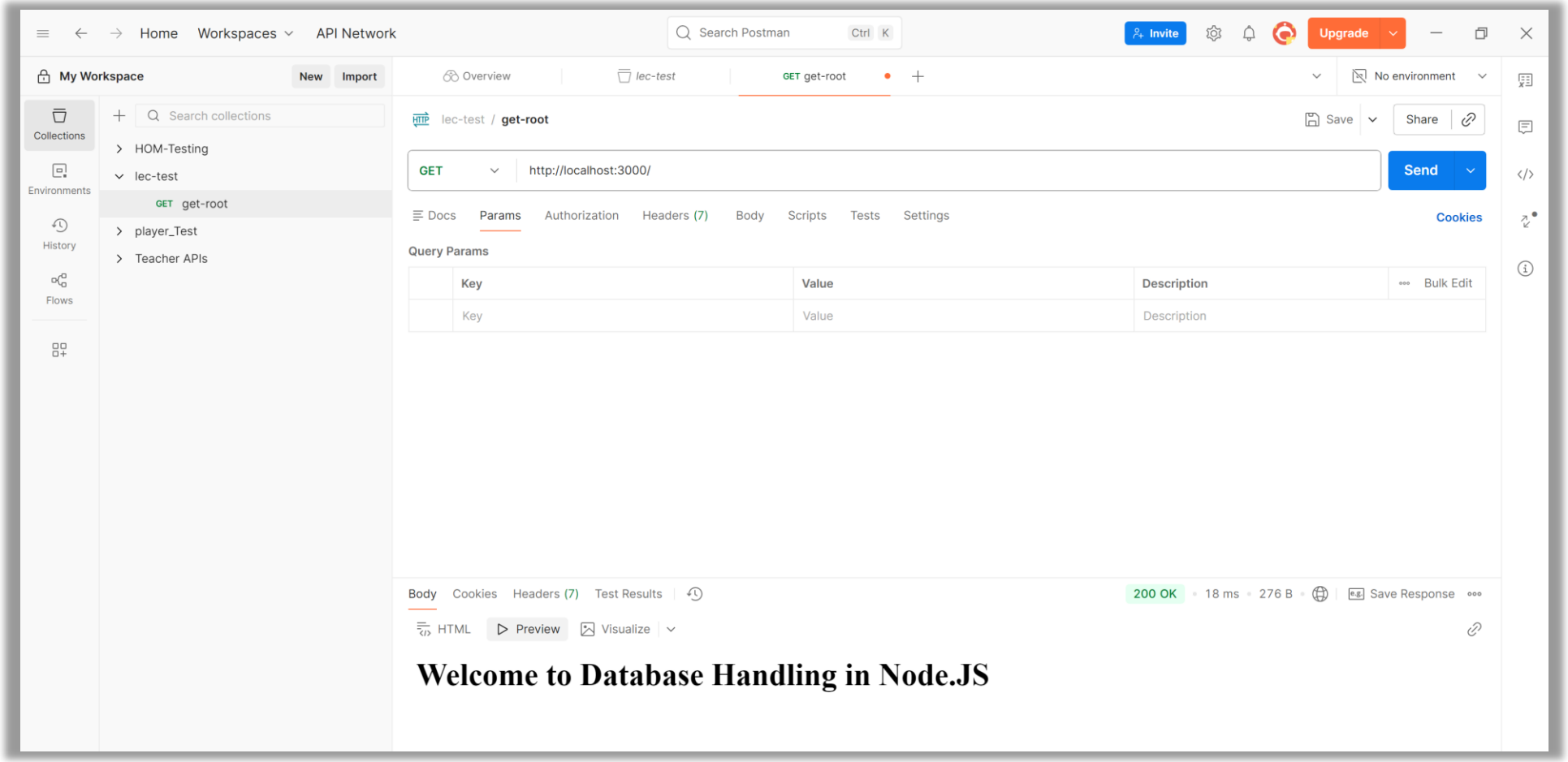
- Let us implement the root route for this app

```
app.get("/", (req, res) => {
  res.send("<h1>Welcome to Database Handling in Node.JS</h1>");
});
```

- Start to listen HTTP request on Server

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Listening on port
${PORT}...`));
```

'/' Route


Home
Workspaces
API Network
Search Postman
Ctrl K
Invite
Upgrade
My Workspace
New
Import
Overview
lec-test
GET get-root
No environment
Collections
Environments
History
Flows
Search collections
HOM-Testing
lec-test
GET get-root
player_Test
Teacher APIs
lec-test / get-root
Save
Share
GET
http://localhost:3000/
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (7)
Test Results
200 OK
18 ms
276 B
Save Response
HTML
Preview
Visualize
Welcome to Database Handling in Node.JS

Run the App

```
malik@DESKTOP-HVT445C MINGW64 /d/Teaching/IS311 Web Development/Lectures/Week 09/code252/dbwebservice
$ npm run dev

> dbwebservice@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Listening on port 3000...
Db is connected successfully...
```

GET '/data/students' Route

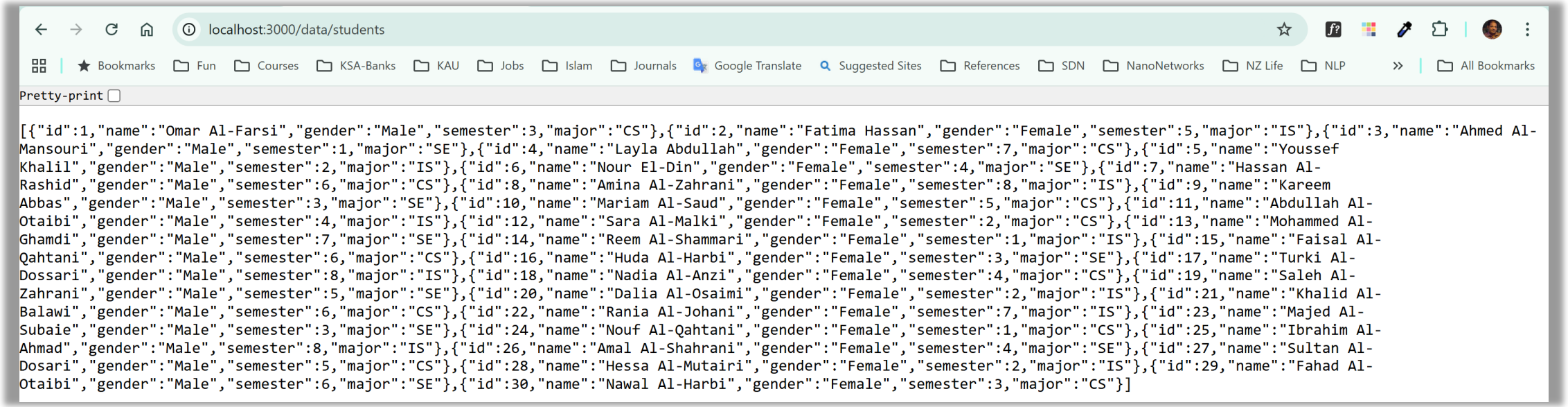
```
app.get("/data/students", (req, res) => {
  // writing query to execute in MySQL
  let sqlQuery = "SELECT * FROM students";
  // executing the query in MySQL and getting back results
  dbCon.query(sqlQuery, (error, results, fields) => {
    if (error) throw error;
    // sending back the records to client
    res.send(results);
  });
});
```

GET '/data/students' Route Flow

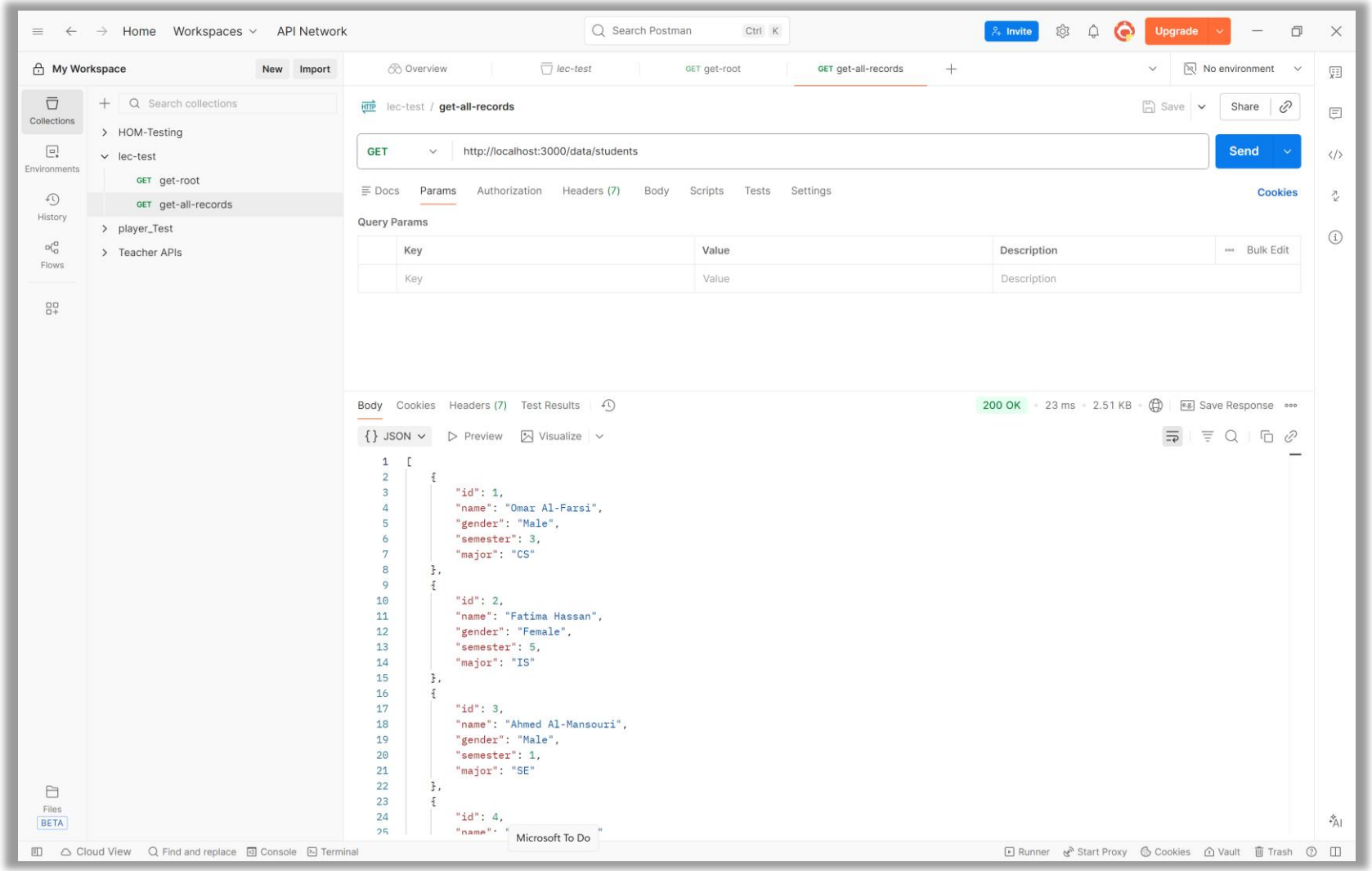
- Browser (Client) send **Get** request on localhost:3000/data/students request to server – index.js file running via nodemon
- Code will accept this request
- Send query to DB Server
- Receive error or data
- Send back records to browser (Client)

```
app.get("/data/students", (req, res) => {
  // writing query to execute in MySQL
  let sqlQuery = "SELECT * FROM students";
  // executing the query in MySQL and getting back results
  dbCon.query(sqlQuery, (error, results, fields) => {
    if (error) throw error;
    // sending back the records to client
    res.send(results);
  });
});
```

GET '/data/students' Route



GET '/data/students' Route

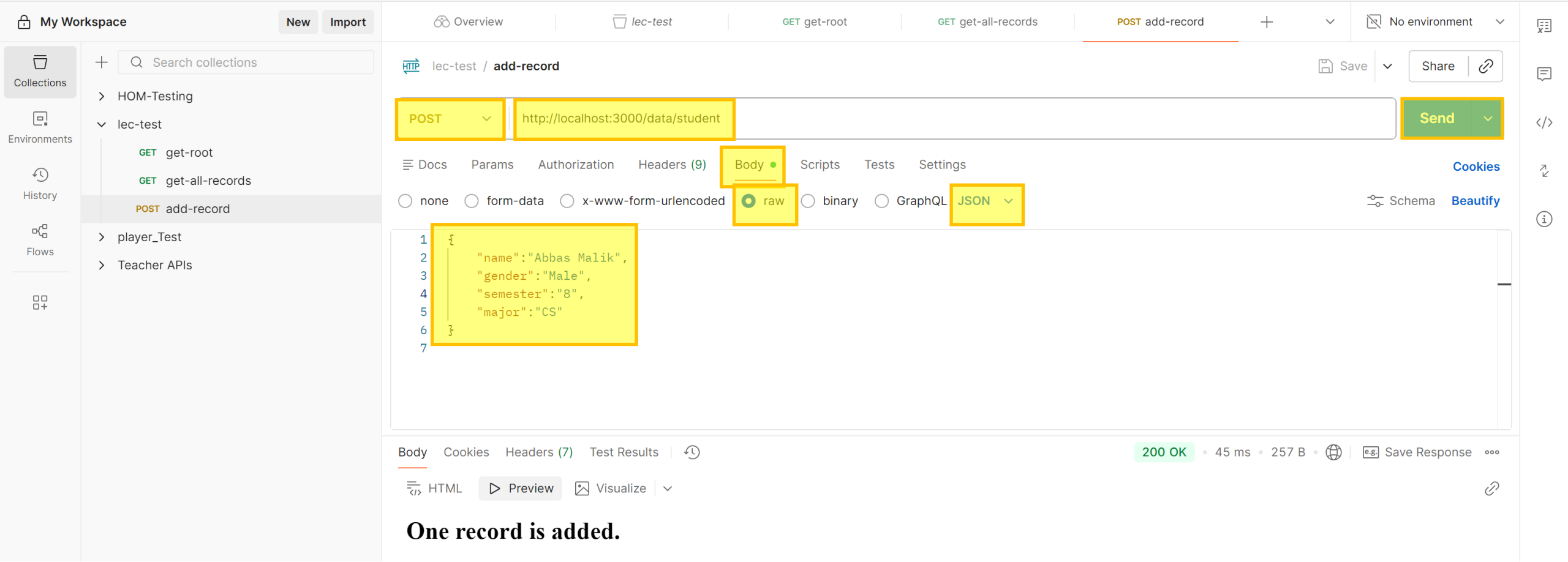


POST '/data/student' Route

```
app.post("/data/student", (req, res) => {
  const student = {
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `INSERT INTO students (name, gender, semester, major) VALUES
      ('${student.name}', '${student.gender}', ${student.semester}, '${student.major}')`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is added.</h2>");
  });
});
```

```
// setting up app to receive JSON in request body
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

POST '/data/student' Route

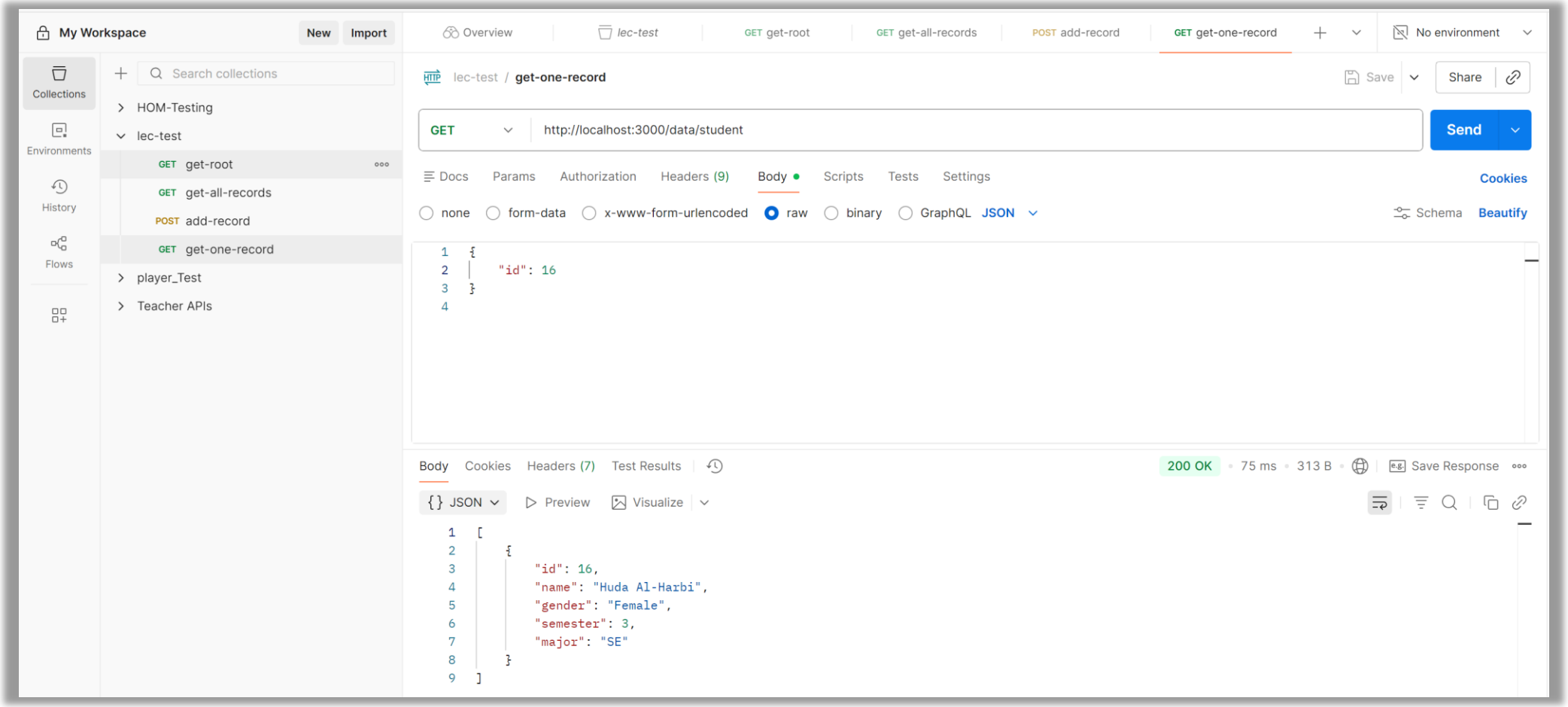


GET '/data/student' Route

- Read only one record from Database Table

```
app.get("/data/student", (req, res) => {
  let sqlQuery = `SELECT * FROM students WHERE id=${req.body.id}`;
  dbCon.query(sqlQuery, (error, results, fields) => {
    if (error) throw error;
    res.send(results);
  });
});
```

GET '/data/student' Route

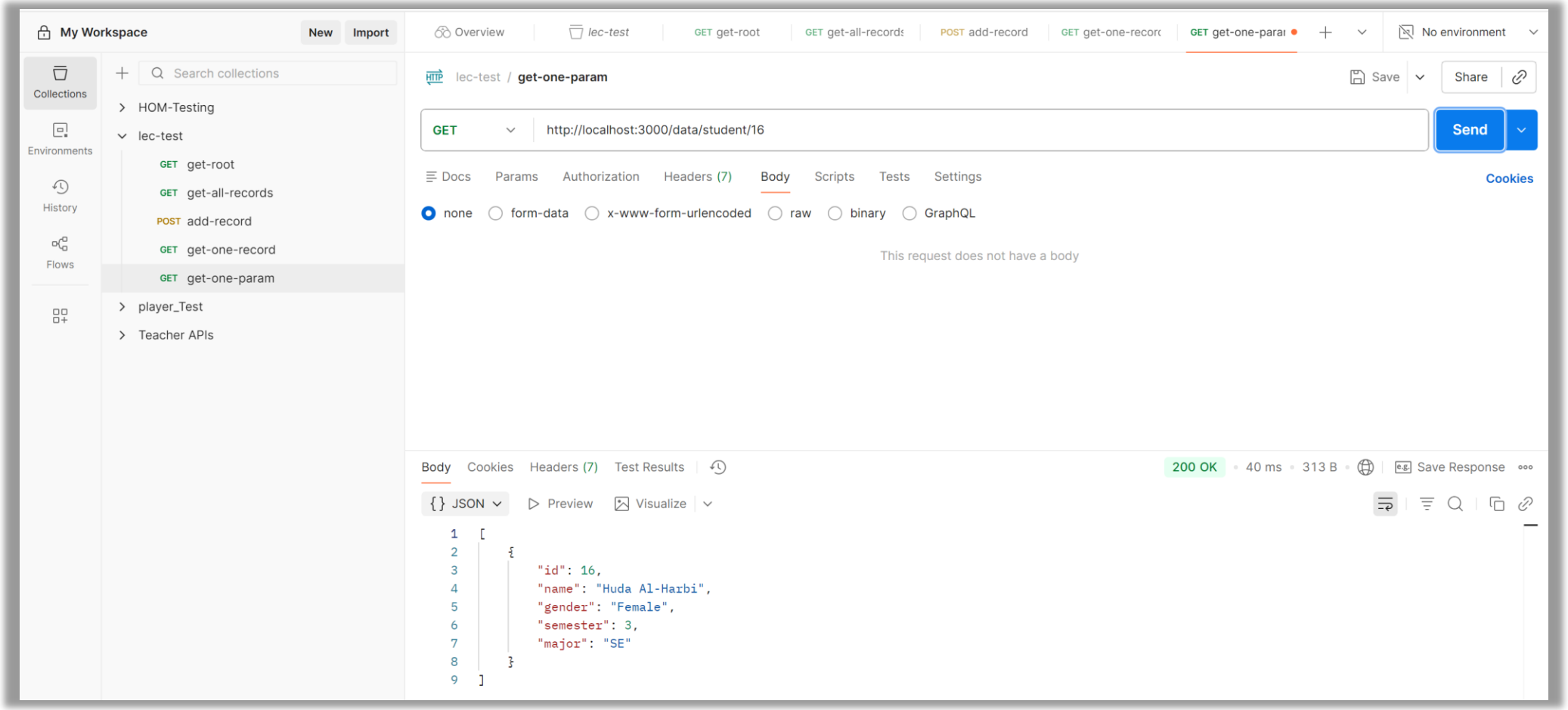


GET '/data/student/id' Route

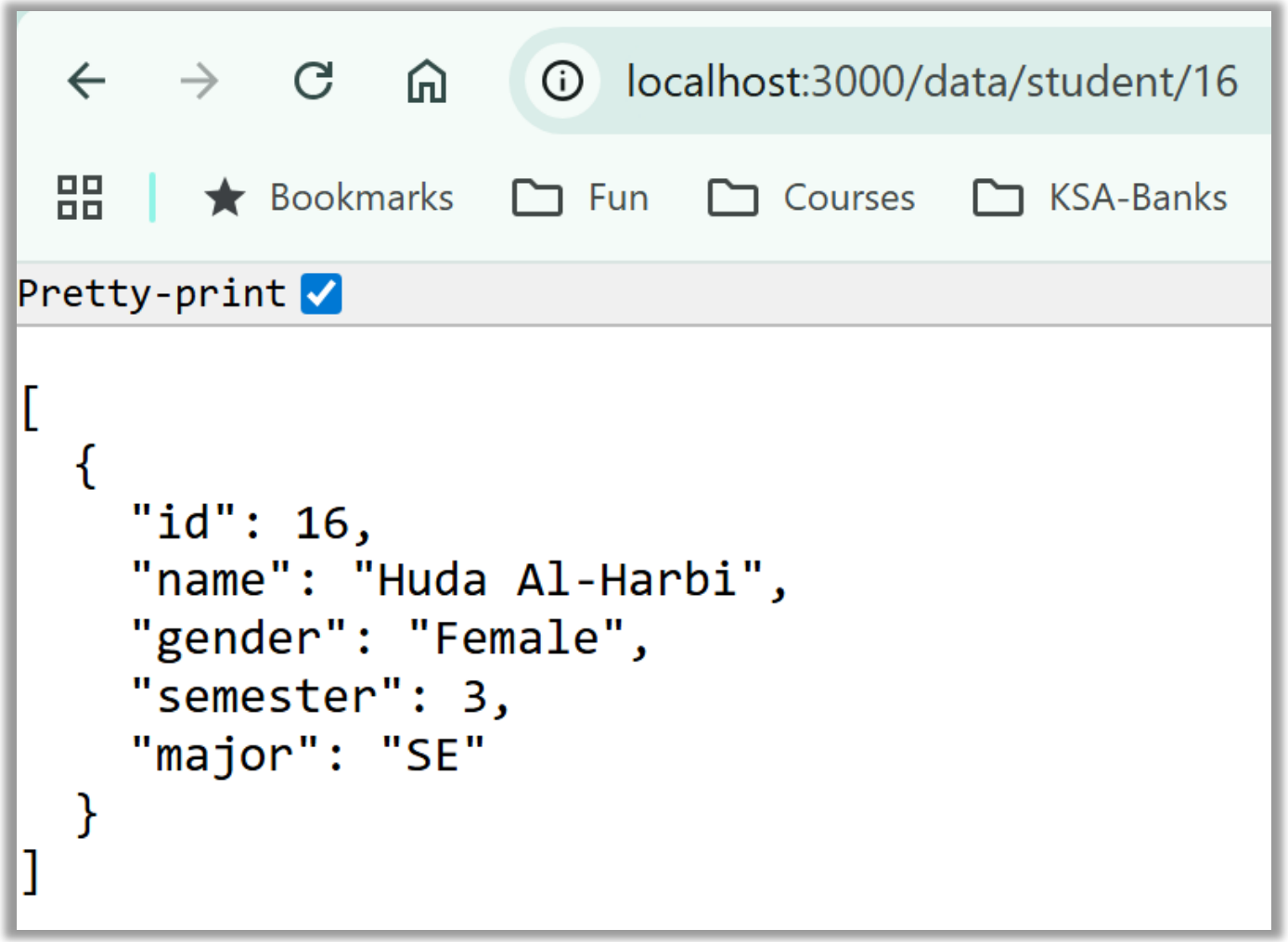
- Read only one record – id as param

```
app.get("/data/student", (req, res) => {
  let sqlQuery = `SELECT * FROM students WHERE id=${req.params.id}`;
  dbCon.query(sqlQuery, (error, results, fields) => {
    if (error) throw error;
    res.send(results);
  });
});
```

GET '/data/student/id' Route



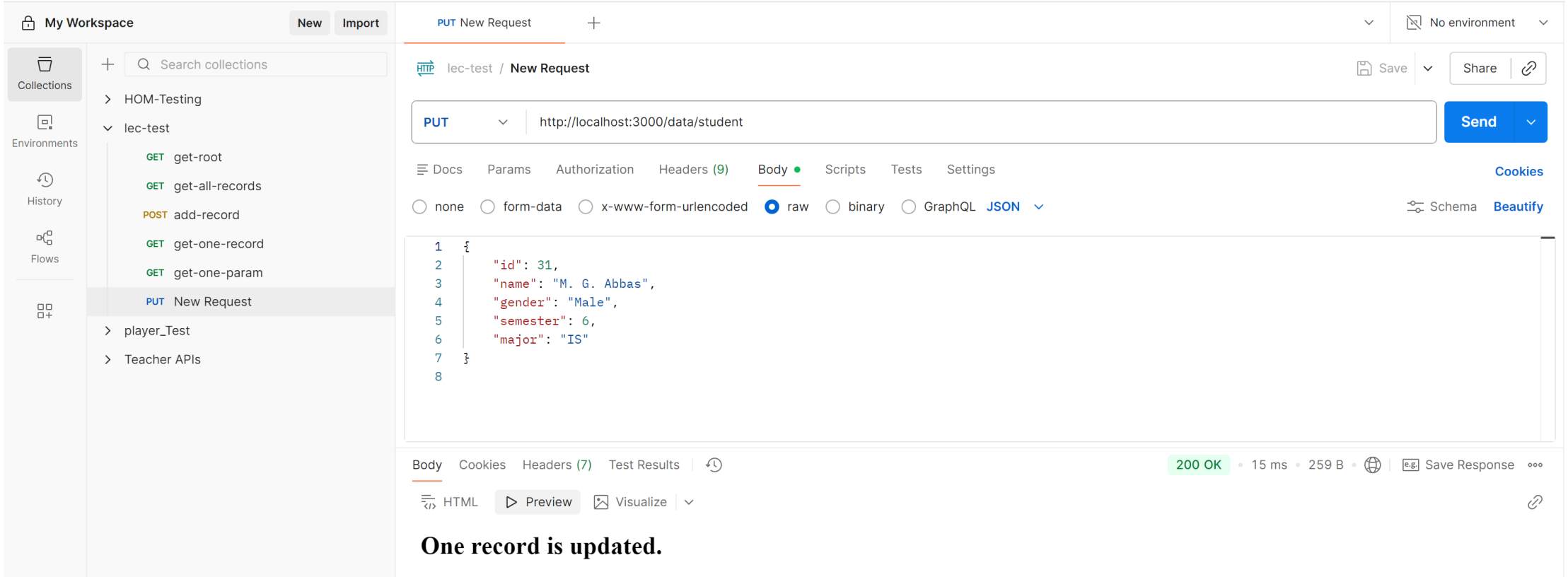
GET '/data/student/id' Route



PUT '/data/student' Route

```
app.put("/data/student", (req, res) => {
  const student = {
    id: req.body.id,
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `UPDATE students
    Set
    name = '${student.name}',
    gender = '${student.gender}',
    semester = ${student.semester},
    major = '${student.major}'
    WHERE id = ${student.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is updated.</h2>");
  });
});
```

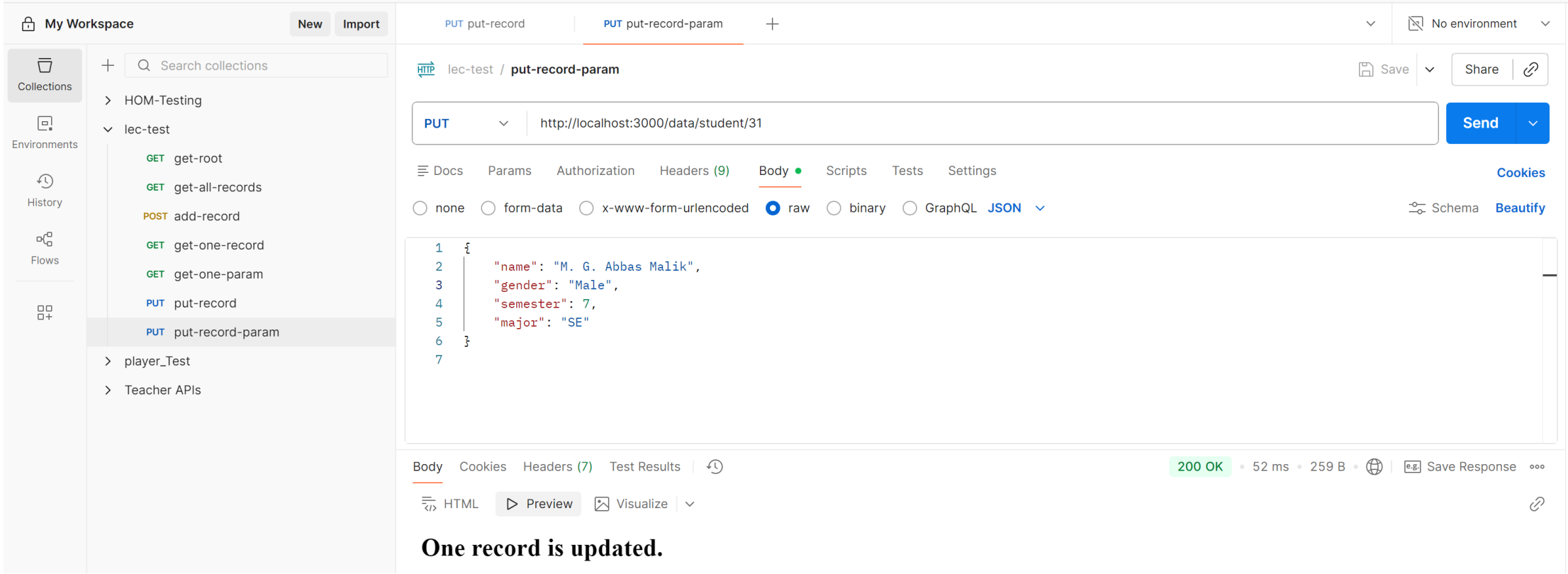
PUT '/data/student' Route



PUT '/data/student/id' Route

```
app.put("/data/student/:id", (req, res) => {
  const student = {
    id: req.params.id,
    name: req.body.name,
    gender: req.body.gender,
    semester: req.body.semester,
    major: req.body.major,
  };
  let sqlQuery = `UPDATE students
    Set
    name = '${student.name}',
    gender = '${student.gender}',
    semester = ${student.semester},
    major = '${student.major}'
    WHERE id = ${student.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is updated.</h2>");
  });
});
```

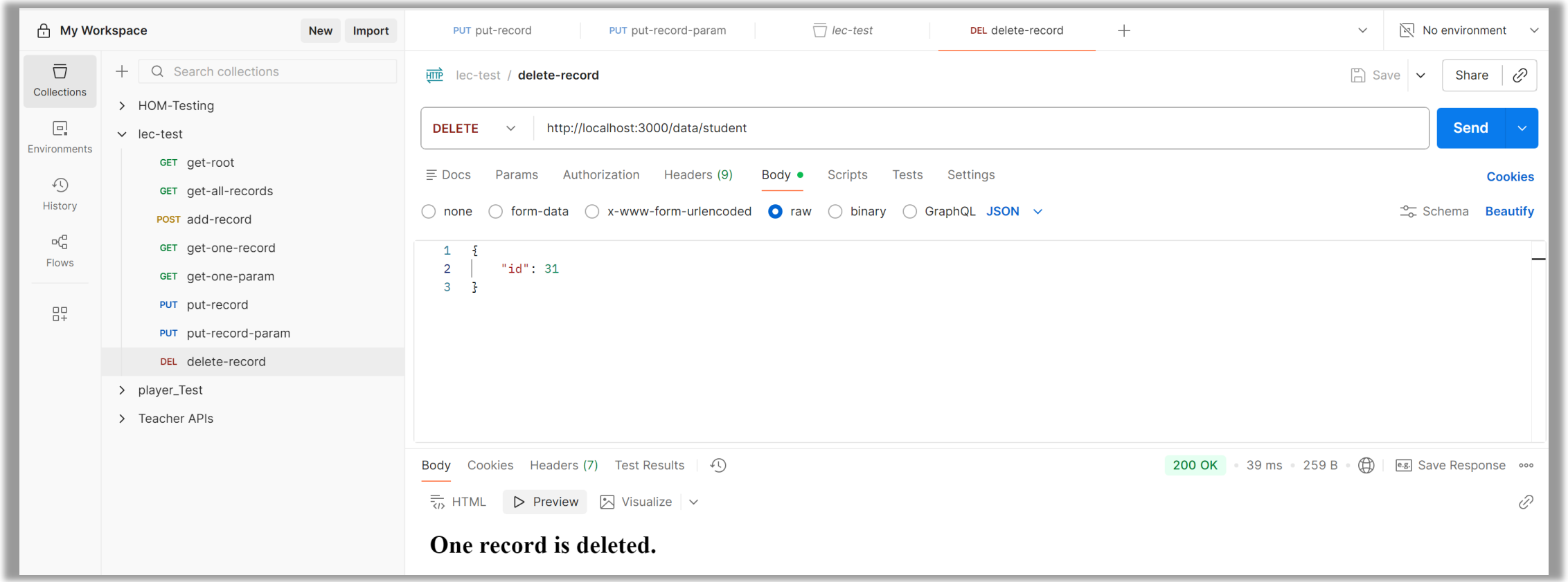
PUT '/data/student/id' Route



DELETE '/data/student' Route

```
app.delete("/data/student", (req, res) => {
  let sqlQuery = `DELETE FROM students
    WHERE id = ${req.body.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is deleted.</h2>");
  });
});
```

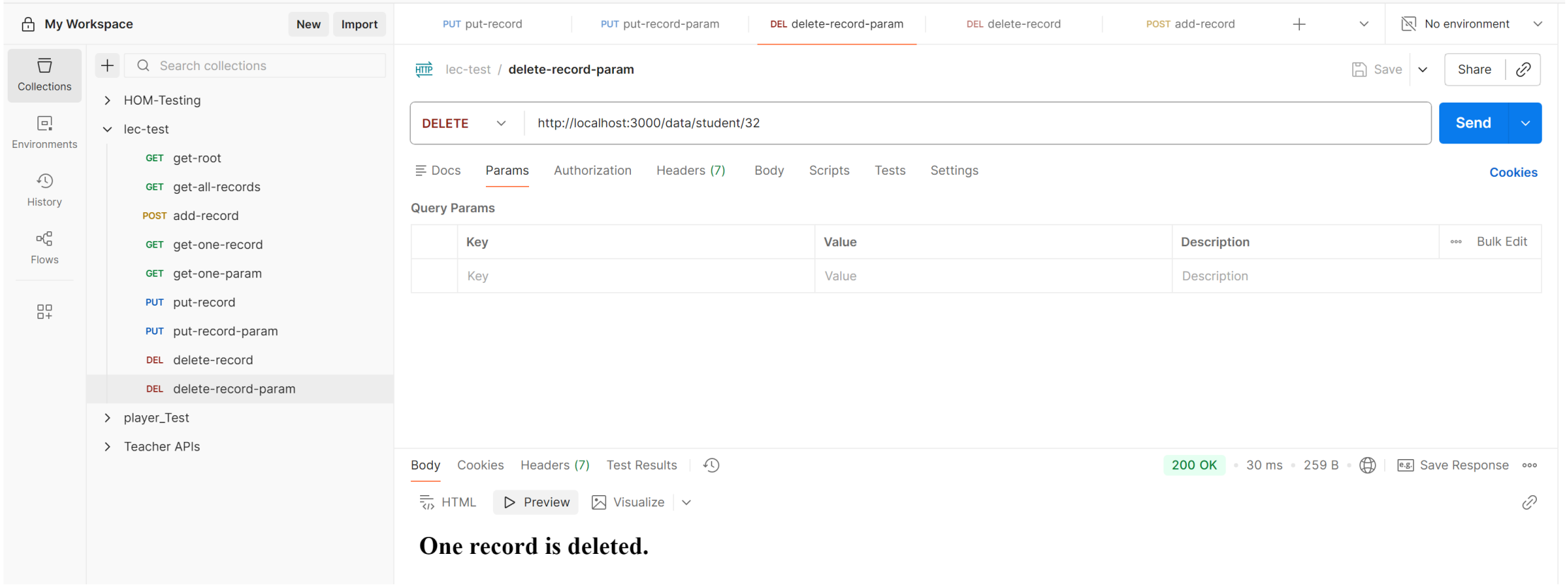
DELETE '/data/student' Route



DELETE '/data/student/id' Route

```
app.delete("/data/student/:id", (req, res) => {
  let sqlQuery = `DELETE FROM students
    WHERE id = ${req.params.id}`;
  dbCon.query(sqlQuery, (error, results) => {
    if (error) throw error;
    res.send("<h2>One record is deleted.</h2>");
  });
});
```

DELETE '/data/student/id' Route



HTTP Request Status

Complete List

- HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

1. Informational responses (100 – 199)
2. Successful responses (200 – 299)
3. Redirection messages (300 – 399)
4. Client error responses (400 – 499)
5. Server error responses (500 – 599)

Set HTTP Request Status

```
app.get("/data/students", (req, res) => {
  // writing query to execute in MySQL
  let sqlQuery = "SELECT * FROM students";
  // executing the query in MySQL and getting back results
  dbCon.query(sqlQuery, (error, results, fields) => {
    if (error) throw error;
    // sending back the records to client
    res.status(200).send(results);
  });
});
```

```
res.status(200);
res.send(results);
```

Set HTTP Request Status

```
app.get("/data/student", (req, res) => {
  let sqlQuery = `SELECT * FROM students WHERE id=${req.body.id}`;
  dbCon.query(sqlQuery, (error, results, fields) => {
    if (error) {
      res.status(500).send({ message: "Internal Server Error.." });
    }
    res.status(200).send({ success: true, data: results[0] });
  });
});
```

HTTP Request Labwork

- Create GET, PUT, POST and DELETE for Course with validations
  - ID is a number, autogenerated and primary key,
  - Name is required, string and at least 5 character long.
  - Semester is between 1 and 8 only
  - Year can be between 1 to 4 only
  - Major can have values: CS, IS or SE
- Structure of Course is:

```
course {
    id: Number,
    name: String,
    code: String,
    semester: Number,
    major: String,
    year: Number,
}
```

Data Validation

- We can write a code to validate the data send by the use via HTTP request
- Security best practice is always validate the user data
- Cannot trust the user about the data
- Status: 400
  - Means bad request

Data Validation

Getting Started

```
if(!req.body.name || req.body.name.length < 3)
    return res.status(400)
    .send('Not a valid data: Name is missing or nor 3 character long!!!');
```

JOI - Node Package that will make data validation easier

```
npm i joi
```

Data Validation with JOI

- Import this package in your code

```
const Joi = require('joi');
```

- Build validation rules using Joi

```
const nameVlidationSchema = Joi.object({
  name: Joi.string().min(3).required(),
});
```

More on data validation rules form JOI

Data Validation with JOI

- Validate with Joi

```
const result = nameVlidationSchema.validate(req.body);
```

- Log this result

```
Listening on port 3000...
{ value: { name: 'M. Rizwan' } }
```

Success

```
Listening on port 3000...
{
  value: {},
  error: [Error [ValidationError]: "name" is required] {
    _original: {},
    details: [ [Object] ]
  }
}
```

Data Validation with JOI

- Validatation Error with Joi

```
console.log(result.error);
```

```
[Error [ValidationError]: "name" is required] {
  _original: {},
  details: [
    {
      message: '"name" is required',
      path: [Array],
      type: 'any.required',
      context: [Object]
    }
  ]
}
```

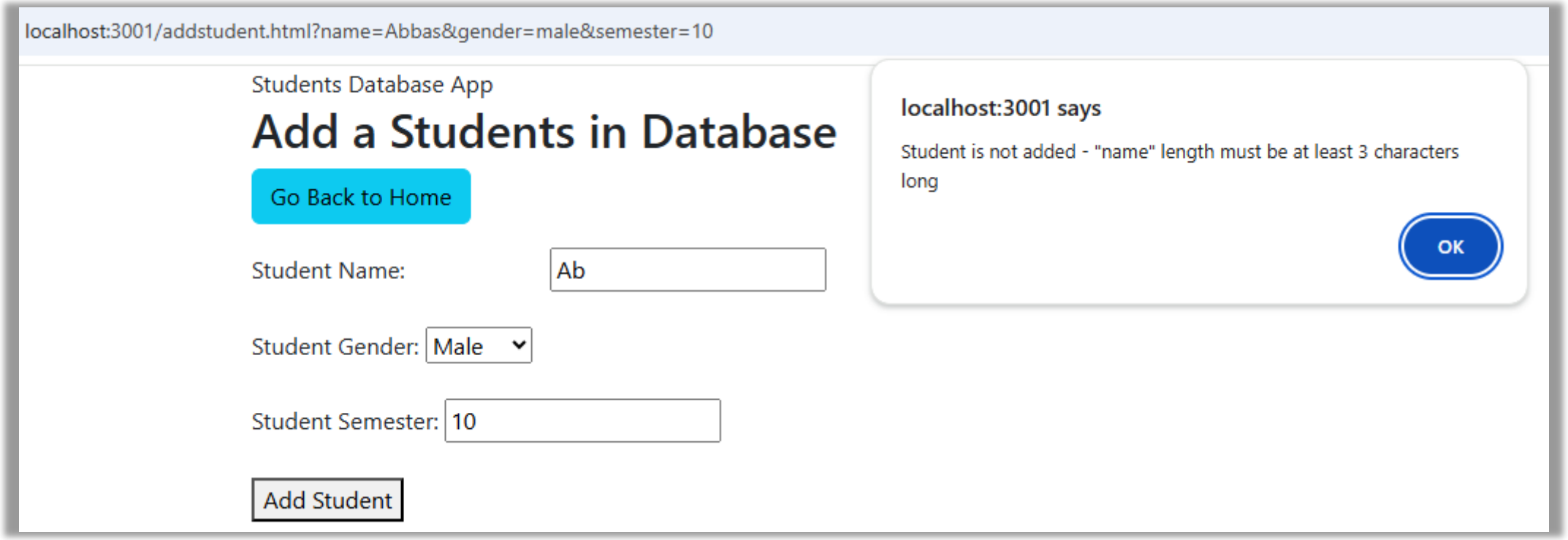
```
console.log(result.error.details[0].message);
```

Multiple Field Validation

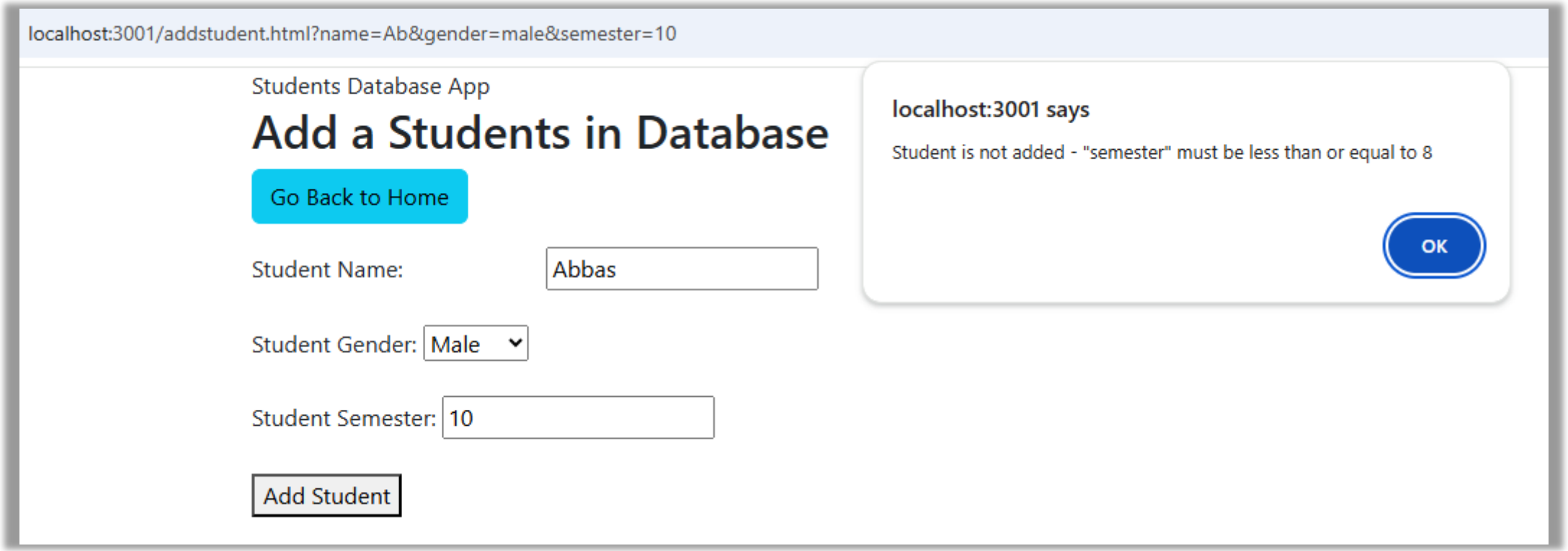
```
const studentValidationSchema = Joi.object({
  name: Joi.string().min(3).max(50).required(),
  gender: Joi.string().required(),
  semester: Joi.number().min(1).max(8).required(),
});
```

```
// validating the student record using JOI
const { error } = studentValidationSchema.validate(studentInfo);

if (error) {
  // one or more validation(s) have failed
  return res.send({ result: "failure", message: error.details[0].message });
}
```

Multiple Field Validation



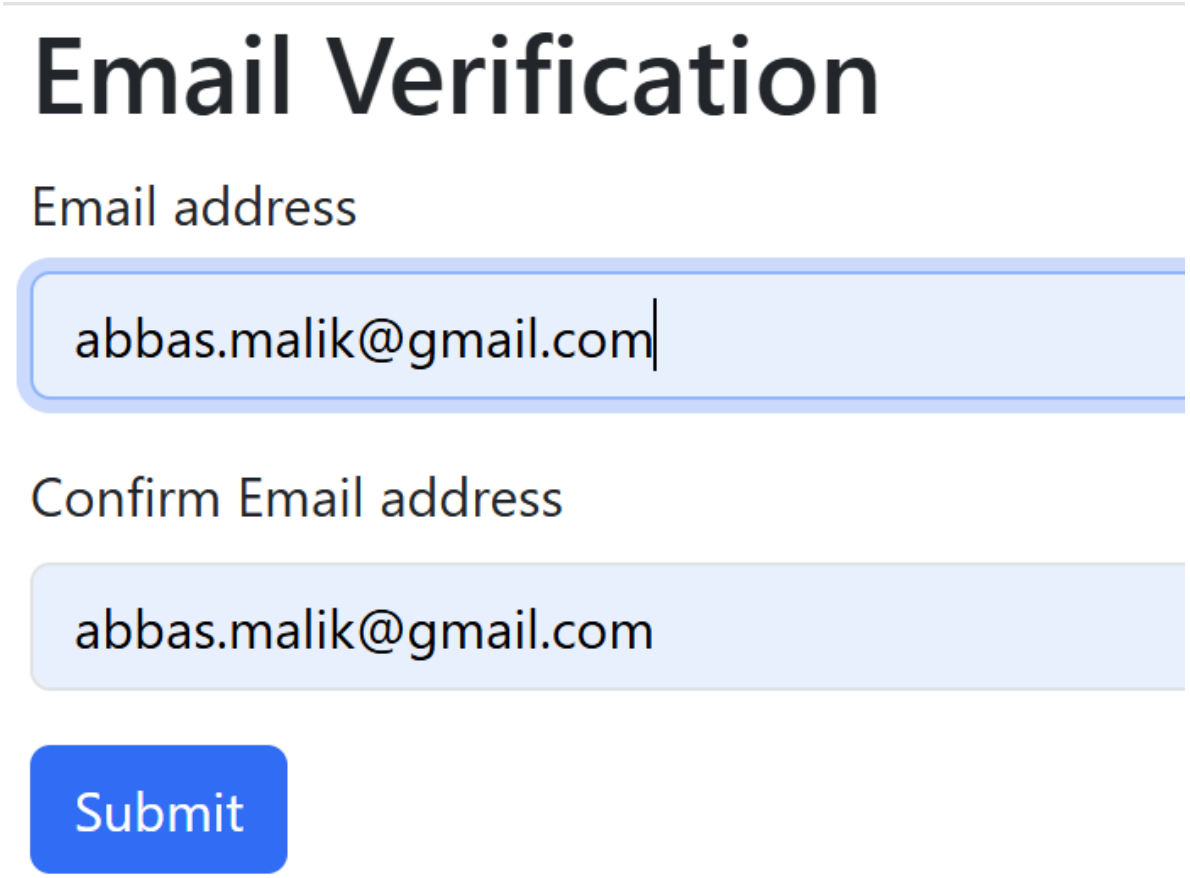
Multiple Field Validation



POST with Data Validation

```
const studentValidationSchema = Joi.object({
  name: Joi.string().min(3).max(50).required(),
  gender: Joi.string().required(),
  semester: Joi.number().min(1).max(8).required(),
});

app.post("/data/students", (req, res) => {
  const studentInfo = req.body;
  if (!studentInfo)
    return res.send({ result: "failure", message: "Body is missing" });
  if (!studentInfo.name || !studentInfo.gender || !studentInfo.semester)
    return res.send({
      result: "failure",
      message: "Body does not contain student info",
    });

  // validating the student record using JOI
  const { error } = studentValidationSchema.validate(studentInfo);

  if (error) {
    // one or more validation(s) have failed
    return res.send({ result: "failure", message: error.details[0].message });
  }

  try {
    let sqlQuery = `INSERT INTO students(name, gender, semester)
    VALUES("${studentInfo.name}", "${studentInfo.gender}", ${studentInfo.semester})`;

    dbConn.query(sqlQuery, (error, result) => {
      if (error) throw error;
```

Email and Confirm Email Validation



Email and ConfirmEmail Validation

```
<body>
  <div class="container">
    <div class="email-form">
      <h2 class="form-title">Email Verification</h2>
      <form id="emailForm" class="needs-validation" novalidate>
        <div class="mb-3">
          <label for="email" class="form-label">Email address</label>
          <input type="email" class="form-control" id="email" required
          placeholder="Enter your email"/>
        </div>
        <div class="mb-3">
          <label for="confirmEmail" class="form-label">Confirm Email address</label>
          <input type="email" class="form-control" id="confirmEmail" required
          placeholder="Re-enter your email"
          />
        </div>
        <button type="submit" class="btn btn-primary submit-btn">
          Submit
        </button>
      </form>
    </div>
  </div>
</body>
```

Email and ConfirmEmail Validation

```
const emailValidationSchema = Joi.object({
  email: Joi.string().min(8).max(16).required(),
  confirmEmail: Joi.ref("email"),
});
```

- confirmEmail must be present and equal to email.
- Validate that two fields are same

Learn more about JOI and Validation

- Tutorial 1
- Tutorial 2

Reference

- https://www.npmjs.com/package/mysql
- https://www.w3schools.com/nodejs/nodejs_mysql.asp
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status